Open in Colab

Build and Deploy a Natural Language Processing (NLP) Model to Classify Sentiments about Amazon's Prime Videos

- **Group:** Group 1 - Edwin George, Moses Musyoki, Nelima Wanyama, Spencer Lugalia, Wambui Munene
- **Student pace:** DSPT08
- **Scheduled project review date/time:** 31/03/2025 23.59 Hours
- **Instructor name:** Samuel Karu

Project Overview

This project focuses on performing sentiment analysis on user reviews related to Amazon Prime Video, leveraging a robust dataset sourced from the Amazon Reviews Dataset compiled by McAuley Lab in 2023. The goal is to derive valuable insights into user sentiment, addressing patterns of positive, neutral, and negative feedback. Understanding sentiment trends can help uncover critical insights into customer satisfaction and areas for improvement.

Amazon Prime Video, launched in 2006 as Amazon Unbox, has evolved significantly over the years. Initially, it was a platform for purchasing and renting digital content. In 2011, it was rebranded as Amazon Instant Video and began offering streaming services as part of the Amazon Prime membership. By 2014, it expanded globally and started producing original content, such as the award-winning series Transparent. Today, Prime Video is a major player in the streaming industry, offering a mix of licensed content, original programming, and live sports.

Amazon Prime Video competitors are Netflix and Disney+ . In comparison, Netflix, which started as a DVD rental service in 1997, transitioned to streaming in 2007 and quickly became a pioneer in the industry. It gained a competitive edge by investing heavily in original content, starting with House of Cards in 2013. Netflix's global reach and extensive library have made it the most subscribed streaming service worldwide. Disney+, launched in 2019, has rapidly gained traction by leveraging its vast library of Disney, Marvel, Star Wars, and Pixar content. It appeals to families and fans of these franchises, making it a strong competitor in the streaming space

Amazon Prime Video distinguishes itself with features like offline viewing, integration with Amazon's ecosystem, and a focus on diverse content, including international programming.

While Netflix leads in original content and global reach, Prime Video's bundled offering with Amazon Prime and its unique features make it a formidable competitor.

Given this highly competitive environment, Sentiment analysis is an incredibly strategic tool for Amazon Prime Video, and for any streaming service in general for the following reasons:-

1. Understanding Customer Preferences

By analyzing reviews, Amazon Prime Video can identify specific content genres, themes, or features that audiences love or dislike. For instance, if a series receives overwhelmingly positive feedback for its storyline or casting, they can prioritize producing similar content.

2. Improving Content Quality

Feedback from negative sentiment can pinpoint areas of dissatisfaction, such as weak storylines, poor production quality, or technical issues. This information allows Prime Video to improve its offerings and avoid repeated mistakes.

3. Targeted Marketing Strategies

Positive sentiment analysis can help identify which content resonates most with users. These insights can guide marketing campaigns, focusing on promoting highly-rated shows and movies to attract new users and retain existing ones.

4. Competitive Benchmarking

Comparing user sentiment between their platform and competitors can reveal content gaps or features where Netflix or Disney+ excel. This helps Prime Video adapt its offerings to outperform rivals.

In an industry where user preferences evolve rapidly, sentiment analysis provides Amazon Prime Video with actionable intelligence to remain competitive and relevant.

Dataset Description

The raw data was sourced from https://amazon-reviews-2023.github.io/. This is a large-scale Amazon Reviews dataset, collected in 2023 by McAuley Lab, and it includes rich features such as:

1. User Reviews (ratings, text, helpfulness votes, etc.)
2. Item Metadata (descriptions, price, raw image, etc.);
3. Links (user-item / bought together graphs).

This is an open-source repository consisting of millions of Amazon reviews grouped by category. We used the User Reviews and Item Metadata dataset from the Movies_and_TV category. This category has 6.5 million users, 748K items, and 17.3 million ratings and consists of reviews of Amazon Prime videos as well as reviews for physical media (DVDs, Blu-

Rays) and digital purchases or rentals of movie and TV shows available on the Amazon e-commerce platform. In our analysis, we have filtered out all other content except Prime Video, the streaming service owned by Amazon.

The dataset is open-source and readily available. It is stored in compressed JSON format. Due to the large size of the data set, and efficient use of computational resources we have extracted 500,000 rows each from the User Reviews and the Item Metadata datasets. Using the Pandas library, the two data sets have been merged into one using the 'parent_asin' column, which is the parent ID of the product. The resulting dataset after filtering out all other media has slightly over 233K Prime Video reviews.

A cursory glance at the merged database shows significant NaN values in the category, price, and description columns. This may reduce the number of usable reviews for Exploratory Data Analysis, considerably.

Key features of the dataset include:

- Review Text – The content of the review provided by users.
- Star Rating – A numerical rating (e.g., 1-5 stars) given by the user.
- Timestamp – The date and time when the review was posted.
- Product Metadata – Information about the movie or TV show being reviewed (e.g., title, genre, release year).
- User Metadata – Anonymized details about the reviewer, such as review count and verified purchase status.

Using the star rating, a new `sentiment` column will be created; `positive` sentiment will be the reviews with 4 or 5-star ratings, `neutral` will be 3 -star rating while `negative` will be 1 or 2-star ratings.

Problem Statement

In the highly competitive streaming industry, dominated by platforms such as Netflix and Disney+, Amazon Prime Video faces the challenge of maintaining and growing its subscriber base while delivering quality content tailored to audience preferences. Despite being a leading service provider, customer satisfaction, content relevance, and engagement remain critical for staying ahead in a landscape shaped by evolving viewer expectations and high-quality offerings from competitors. To achieve this, Amazon Prime Video must address key issues:

1. **Addressing Negative sentiments**

Negative customer reviews provide a vital lens to understand dissatisfaction and frustration. However, they also present unique challenges for analysis and resolution. Negative reviews often highlight specific pain points, such as:

- Poor content quality (e.g., weak storylines or poor casting decisions).

- Technical issues (e.g., buffering problems or app crashes).
- Unfavorable pricing or subscription-related concerns.

Understanding and addressing these recurring themes can significantly improve user satisfaction and retention.

2. **Differentiating Neutral Sentiment**

Neutral reviews pose a unique challenge. They often lack emotional intensity, making it harder to distinguish dissatisfaction or lukewarm reception from genuine approval. Without proper analysis, significant insights might be overlooked.By differentiating neutral content form positive and negative sentiments, a deep dive analysis of the neutral content can be done to decipher the actual tone and nuances.

3. **Improving Content Strategy**

Customer reviews may contain recurring themes related to genres, storylines, production quality, or casting choices. Identifying these trends is essential for optimizing content creation and curation.

4. **Actionable Business Insights**

Beyond sentiment classification, Prime Video must leverage insights to inform marketing, pricing models, and recommendation systems.By expanding the scope of sentiment analysis, focusing on nuanced insights from bigrams, trigrams, and parts of speech (adjectives and adverbs), can help uncover hidden trends in user sentiment and drive actionable changes across content, engagement strategies, and user experience.

Business Objectives

With this understanding of the streaming business and the challenges there to, out analysis and modeling will seek to achieve the following objectives:-

1. **Understand sentiment by genre**

Understanding sentiment `by genre` involves analyzing user feedback, reviews, and ratings to determine how audience emotions vary across different genres of content on Prime Video. By categorizing sentiment—such as positive, negative, or neutral—within genres like drama, comedy, action, or documentaries, Prime Video can gain insights into how specific types of content resonate with viewers. This could help identify genre-specific patterns in audience preferences, emotional engagement, or even potential areas for improvement in content offerings, enhancing use satisfaction.

2. **Improving Customer Retention**

Identify drivers of `positive sentiment` to ensure customer satisfaction and retention - these may include genres, titles, cast, production quality etc. After identifying positive

sentiments, a deep dive into these sentiments can uncover audience preferences and identify factors thta drive the sentiment. These drivers may include specific genres that resonate with viewers, popular titles that generate excitement, or star-studded casts that captivate audiences. Additionally, production quality, storytelling, and even the diversity of content offerings could play a role. By aligning strategy with these insights, Prime Video can tailor content recommendations, promotional efforts, and overall user experience to strengthen customer loyalty, foster emotional connections, and reduce churn rates

3. **Enhancing Product Offerings**

Enhancing product offerings through sentiment analysis requires a targeted approach to understand which features or content types may be falling short in meeting customer expectations. By evaluating `negative sentiment` from user feedback, ratings, and reviews, recurring themes or elements that receive criticism — whether it's a particular genre, title, cast member, or technical feature like streaming quality can be identified. For example, viewers might express dissatisfaction with overly complex storylines in a specific genre or lack of diversity in casts.This proactive approach ensures that Prime Video stays ahead of customer needs, enhancing the overall user experience and fostering brand loyalty.

4. **Refine Content and Service Differentiation**

`Neutral sentiment` often highlights areas where customer experiences are neither strongly positive nor negative.This class provides a subtle but critical opportunity to identify areas for incremental improvement, ensuring that "average" feedback doesn't turn into dissatisfaction over time. By analyzing neutral reviews, Amazon Prime Video can:-

- Improve features or offerings to transform neutral experiences into positive ones.
- Identify aspects of content or service that users find adequate but unremarkable.
- Gauge potential gaps between customer expectations and the service delivered, helping to fine-tune content strategies and user experience enhancements.

Requirements to Meet Objectives

1. Load the Data

Use Pandas to load the dataset and get a sense of what is in the dataset by visually inspecting the data.

2. Perform Data Cleaning with nltk

- Use Regular Expressions (REGEX) to remove irrelevant information such as punctuations and numbers
- Convert all text to lowercase to ensure uniformity

- Apply lemmatization to reduce words to their base forms for consistent analysis and reducing complexity
- Remove stop words (common words that typically do not carry significant meaning such as "the," "is," "in," "and," etc.). This helped in focusing on more meaningful words in the text, leading to better performance of NLP models.
- Tokenize the cleaned text

3. Perform Exploratory Data Analysis and Visualization

- Analyze the positive and negative sentiments by company.
- Analyze the distribution of sentiment labels (positive, negative,neutral) using bar charts and value counts to understand class balance.
- Visualize sentiment distributions and highlight key n-grams and POS-tagged patterns.
- Generate insights using heatmaps, word clouds, and bar charts.

4. Vectorize the text data with TFidfVectorizer

- All data must be in numeric form in order to fit a scikit-learn model. We will use Term Frequency-Inverse Document Frequency (TF-IDF) vectorizer from sklearn.feature_extraction.text to convert the text data into a vectorized format.Using TF-IDF is important because it effectively weighs the significance of words in a document relative to the entire dataset, helping to distinguish relevant terms from common ones.

5. Iteratively Build and Evaluate Baseline and Advanced Models

- Using Pipelines, build and iteratively tune baseline SVM and Multinomial Naive Bayes Models
- Build and Train one or more Advanced models and compare the results with those of the tuned baseline models

6. Evaluation

Evaluate model performance using the following metrics:

- Classification_report from Scikit-learn : This metric provides a convenient way to generate detailed performance metrics for classification tasks. It provides a summary of key metrics for each class, including accuracy, precision, recall, and F1-score
- confusion_matrix: It provides a visual summary of the prediction results by showing the count of true positives, true negatives, false positives, and false negatives.

7. Deploy the Model

Build the Flask API

- Set up endpoints to serve sentiment and summarization models, ensuring input validation and error handling.
- Load trained models for efficient processing.

Front-End Integration

- Connect the Flask API to an interactive dashboard for user-friendly review analysis, offering features like sentiment visualization and text summarization.

Deploy the Application

- Use a deployment platform like AWS to host the Flask API and dashboard (Optional)

Testing & Monitoring

- Test functionality thoroughly and implement monitoring tools to track performance and address issues.
- Flask API to serve sentiment and summarization models.

8. Next Steps

Using the results obtained from the evaluation process make recommendations to Amazon Pride on how they can leverage Sentiment Analysis for customer retention, improve product offerings and service differentiation.

Key Technologies Used

- Python (Pandas, NumPy, NLTK, Scikit-Learn, TensorFlow, PyTorch)
- NLP Models (TF-IDF, LSTM, BERT)
- Machine Learning (MultinomialNB, SVM, Random Forest)
- Deep Learning (RNN, LSTM)
- Data Visualization (Matplotlib, Seaborn, WordCloud)
- Flask (For web deployment)
- Cloud Deployment (Optional: AWS, Heroku, or Streamlit)

Expected Outcomes

- Accurate sentiment classification of Amazon Prime Video reviews.
- Concise review summaries to assist in decision-making.
- Scalable and deployable Flask-based API for real-time analysis.

Future Enhancements

- Aspect-Based Sentiment Analysis – Identify sentiment for specific aspects (e.g., content quality, streaming experience).
- Emotion Detection – Extend sentiment analysis to classify emotions like joy, anger, and frustration.
- Personalized Recommendations – Suggest movies/shows based on sentiment and user preferences.

1.1 Load and Clean the Dataset

In [1]:
```
# from google.colab import drive
# drive.mount('/content/drive')
```

In [2]:
```
# Import the necessary libraries for data analysis, visualization and modeling
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
%matplotlib inline
import warnings
warnings.filterwarnings('ignore')
import re
from bs4 import BeautifulSoup
from collections import Counter
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
from wordcloud import WordCloud
import spacy
from spacy import displacy
import joblib

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.model_selection import GridSearchCV
from sklearn.utils import resample

import matplotlib.pyplot as plt
import seaborn as sns
```

In [3]:
```
# nltk.download('wordnet')
# nltk.download('stopwords')
# nltk.download('punkt_tab')
# nltk.download('averaged_perceptron_tagger')
# nltk.download('averaged_perceptron_tagger_eng')
```

In [4]:
```
# Load the data as a DataFrame and display the first 5 columns
video_df = pd.read_csv("prime_videos.csv", encoding='ISO-8859-1')
```

In [5]:
```
# check the first 3 rows
video_df.head(3)
```

Out[5]:

	rating	title_x	text	asin	parent_asin	timestamp	verified_purchase	mair
0	5	Five Stars	Amazon, please buy the show! I'm hooked!	B013488XFS	B013488XFS	2015-08-24 03:07:17.000	True	P
1	5	Five Stars	My Kiddos LOVE this show!!	B00CB6VTDS	B00CB6VTDS	2016-04-19 21:16:50.000	True	P
2	5	What Love Is...	...isn't always how you expect it to be, but w...	B001H1SVZC	B001H1SVZC	2020-05-28 04:13:47.074	True	P

In [6]:
```
# check the last 3 rows to ensure uniformity
video_df.tail(3)
```

Out[6]:

	rating	title_x	text	asin	parent_asin	timestamp	verified_purcha
233607	5	Another Instant Will Ferrell Classic!!!	Will Ferrell has always been one of my favorit...	B00V9US0TI	B00V9US0TI	2015-03-28 02:27:19.000	Fal
233608	5	Scary Movie 5 is Easily One of the Best in the...	I grew up with the Scary Movie franchise. I've...	B00EAZL6HC	B00EAZL6HC	2013-08-20 02:43:44.000	Fal
233609	4	Pretty Good!!!	This movie was a lot better than I had expecte...	B002VXKYCS	B002VXKYCS	2009-12-31 01:25:26.000	Fal

In [7]:

```
# check the shape of the data
video_df.shape

print(f"This data set consists of {video_df.shape[0]} rows")
print(f"This data set consists of {video_df.shape[1]} columns")
```

```
This data set consists of 233610 rows
This data set consists of 14 columns
```

In [8]:

```
# Get column names
video_df.columns
```

Out[8]:

```
Index(['rating', 'title_x', 'text', 'asin', 'parent_asin', 'timestamp',
       'verified_purchase', 'main_category', 'title_y', 'average_rating',
       'rating_number', 'description', 'price', 'categories'],
      dtype='object')
```

In [9]:

```
#check for missing values
video_df.isna().sum()
```

Out[9]:
```
rating                   0
title_x                 24
text                    31
asin                     0
parent_asin              0
timestamp                0
verified_purchase        0
main_category            0
title_y             199810
average_rating           3
rating_number            3
description         199810
price               209445
categories          199810
dtype: int64
```

The dataset has alot of missing values in the title_y, description, price and categories columns. This means the title of the movie, its genre, itle description and price are not included. These are necessary for EDA.

In [10]:
```
video_df.duplicated().sum()
```

Out[10]:
```
538
```

In [11]:
```
# Rename and drop some columns for better readability and analysis

video_df = video_df.rename(columns={'text':'review_text','title_x':'rating_desc', '
video_df = video_df.drop(columns=['asin','parent_asin','rating_number','main_catego

print(video_df.columns)
```

```
Index(['rating', 'rating_desc', 'review_text', 'timestamp',
       'verified_purchase', 'movie_title', 'average_rating', 'description',
       'price', 'categories'],
      dtype='object')
```

Columns have been renamed, and columns that are not necessary for analysis have been dropped

In [12]:
```
# Get column attributes
video_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 233610 entries, 0 to 233609
Data columns (total 10 columns):
 #   Column             Non-Null Count   Dtype
---  ------             --------------   -----
 0   rating             233610 non-null  int64
 1   rating_desc        233586 non-null  object
 2   review_text        233579 non-null  object
 3   timestamp          233610 non-null  object
 4   verified_purchase  233610 non-null  bool
 5   movie_title        33800 non-null   object
 6   average_rating     233607 non-null  float64
 7   description        33800 non-null   object
 8   price              24165 non-null   float64
 9   categories         33800 non-null   object
dtypes: bool(1), float64(2), int64(1), object(6)
memory usage: 16.3+ MB
```

The data types are appropriate for the columns.However, there aresignificant NULL values in the movie_title,description,price and categories columns. Since one of our objectives is to analyze sentiment by genre, we need the categories column not to be null. We will drop all the rows in the categories columns with NULL values.

In [13]:

```
# replave NaN in the 'price' column with the mean
# drop all rows with NaN values in the 'categories','description' and 'movie_title'

video_df['price'].fillna(video_df['price'].mean(), inplace=True)

video_df = video_df.dropna(subset=['categories','description','movie_title'])

# check for the info
video_df.info()

print("\nvideo_df.shape:", video_df.shape)
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 33800 entries, 0 to 233601
Data columns (total 10 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   rating             33800 non-null  int64
 1   rating_desc        33798 non-null  object
 2   review_text        33798 non-null  object
 3   timestamp          33800 non-null  object
 4   verified_purchase  33800 non-null  bool
 5   movie_title        33800 non-null  object
 6   average_rating     33799 non-null  float64
 7   description        33800 non-null  object
 8   price              33800 non-null  float64
 9   categories         33800 non-null  object
dtypes: bool(1), float64(2), int64(1), object(6)
memory usage: 2.6+ MB

video_df.shape: (33800, 10)
```

The NULL values from the specified columnsand the NaN in the `price` column have been updated with the average price. However, there are still a few lingering NaN values, which will be dropped in the cell below

In [14]:
```
# drop the remaining nulls
video_df = video_df.dropna()

# check for the info
video_df.info()

print("\nvideo_df.shape:", video_df.shape)
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 33796 entries, 0 to 233601
Data columns (total 10 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   rating             33796 non-null  int64
 1   rating_desc        33796 non-null  object
 2   review_text        33796 non-null  object
 3   timestamp          33796 non-null  object
 4   verified_purchase  33796 non-null  bool
 5   movie_title        33796 non-null  object
 6   average_rating     33796 non-null  float64
 7   description        33796 non-null  object
 8   price              33796 non-null  float64
 9   categories         33796 non-null  object
dtypes: bool(1), float64(2), int64(1), object(6)
memory usage: 2.6+ MB

video_df.shape: (33796, 10)
```

All the NaN values have been dropped.

In [15]:
```
# Remove rows where 'categories' contain '['
video_df = video_df[~video_df['categories'].str.contains('[', regex=False)]
video_df['categories'].value_counts()
```

Out[15]:
```
categories
Suspense                     7518
Comedy                       7507
Drama                        7075
Science Fiction              3945
Documentary                  1604
Fantasy                       802
Action                        783
Unscripted                    753
Historical                    567
Kids                          474
Animation                     411
Horror                        379
Adventure                     325
Special Interest              322
International                 218
Young Adult Audience          203
Romance                       141
Western                       132
Sports                         96
Arts                           86
Faith and Spirituality         64
LGBTQ                          54
Music Videos and Concerts      40
Military and War               38
Arthouse                       31
Fitness                        30
Anime                          20
Talk Show and Variety           7
Name: count, dtype: int64
```

In [16]:
```
video_df.duplicated().sum()
```

Out[16]:
```
73
```

In [17]:
```
video_df = video_df.drop_duplicates()
video_df.shape
```

Out[17]:
```
(33552, 10)
```

In [18]:
```
#Change 'timestamp' data type to datetime
video_df['timestamp'] = pd.to_datetime(video_df['timestamp'])

video_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 33552 entries, 0 to 233601
Data columns (total 10 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   rating             33552 non-null  int64
 1   rating_desc        33552 non-null  object
 2   review_text        33552 non-null  object
 3   timestamp          33552 non-null  datetime64[ns]
 4   verified_purchase  33552 non-null  bool
 5   movie_title        33552 non-null  object
 6   average_rating     33552 non-null  float64
 7   description        33552 non-null  object
 8   price              33552 non-null  float64
 9   categories         33552 non-null  object
dtypes: bool(1), datetime64[ns](1), float64(2), int64(1), object(5)
memory usage: 2.6+ MB
```

'timestamp'column is now in a datetime64 format suitable for time series analysis.

In [19]:
```
# Create 'sentiments' column
def classify_sentiment(rating):
    if rating in [1, 2]:
        return 'negative'
    elif rating == 3:
        return 'neutral'
    elif rating in [4, 5]:
        return 'positive'

video_df['sentiment'] = video_df['rating'].apply(classify_sentiment)

video_df['sentiment'].value_counts()
```

Out[19]:
```
sentiment
positive    25094
negative     4917
neutral      3541
Name: count, dtype: int64
```

- The sentiments are `largely Positive` with a `fewer neutral` and `negative` sentiments. It would be of interest to Amazon and other content creators to determine which titles and genres have the most positive, neutral or negative sentiments

In [20]:
```
video_df['verified_purchase'].value_counts()
```

Out[20]:
```
verified_purchase
True     26984
False     6568
Name: count, dtype: int64
```

There appears to be rows where the verifiied purchase is false, meaning the purchase was not verified. We will drop all those rows.

In [21]:
```
# Drop rows where verified_purchase  is false
video_df = video_df.drop(video_df[video_df['verified_purchase']==False].index)
video_df.shape
```

Out[21]:
```
(26984, 11)
```

New shape indicates that the unverified purchases have been dropped

In [22]:
```
video_df.describe()
```

Out[22]:

	rating	timestamp	average_rating	price
count	26984.000000	26984	26984.000000	26984.000000
mean	4.113252	2017-09-06 06:58:19.671578368	4.430559	14.324602
min	1.000000	2007-08-17 05:48:51	1.000000	0.000000
25%	4.000000	2016-01-23 20:04:25.500000	4.300000	10.790000
50%	5.000000	2016-12-29 20:55:58	4.600000	14.570061
75%	5.000000	2019-08-06 17:50:35.622000128	4.700000	14.990000
max	5.000000	2023-03-17 20:13:30.343000	5.000000	81.990000
std	1.266040	NaN	0.434535	5.662028

`rating` and `average_rating` both show a bias towards high values, indicating most content receive favorable evaluations. The time stamp indicates we have data from 2007 to 2023 - a period of over 5 years. We can therefore analyze sentiments over time.

`price` shows a concentration around mid-range values (~14.3), with outliers suggesting a few premium-priced items. The minimum price of 0 is not surprising as Amazom Prime offers a selection of movies and TV shows that are free to watch with a Prime Membership.Additionally, Amazon also has some free content available with ads, which doesn't require a Prime membership.

In [23]:
```
video_df['sentiment'].value_counts()
```

Out[23]:
```
sentiment
positive    20375
negative     3715
neutral      2894
Name: count, dtype: int64
```

There is a significant class balance between the majoority class `positive` and the other classes. This could lead to model bias. We will use modeling algorithms like SVM that handle class imbalnce better and also use other techniques like undersampling to handle this class imbalance.

In [24]:
```
# Make a copy of the cleaned data set before text pre-processing and EDA
data = video_df.copy()
data.head(3)
```

Out[24]:

	rating	rating_desc	review_text	timestamp	verified_purchase	movie_title	average_rat
0	5	Five Stars	Amazon, please buy the show! I'm hooked!	2015-08-24 03:07:17	True	Sneaky Pete	
1	5	Five Stars	My Kiddos LOVE this show!!	2016-04-19 21:16:50	True	Creative Galaxy	
14	4	Four Stars	It was pretty good. Had some pretty comical sc...	2016-09-16 18:39:49	True	Seven Girlfriends	

In [25]:
```
data = data.reset_index(drop=True)
```

1.2 Text Transformations

In this section, we will prepare the data dataframe for `EDA` and `Modeling`.

- Join the text in rating_desc and review_text into a new column called reviews. This will add even more context to the reviews
- We will drop columns that have no analytical values like description, verified_purchase etc.
- Then we will prepare the review_text feature column for vectorization,This will involve:-
  - removing unnessary text and symbols- html tags, numbers, punctuation and symbols that carry no meaning to the sentiments.
  - convert all text to lowercase to ensure uniformity
  - apply lemmatization to reduce words to their base forms for consistent analysis and reducing complexity
  - remove stop words (common words that typically do not carry significant meaning such as "the," "is," "in," "and," etc.).
  - tokenize the cleaned text into individual words

In [26]:
```
# Join the text in 'rating_desc' and 'review_text' into a new column called 'review
data['reviews'] = data['rating_desc'] + '  ' + data['review_text']
data.head(2)
```

Out[26]:

	rating	rating_desc	review_text	timestamp	verified_purchase	movie_title	average_rati
0	5	Five Stars	Amazon, please buy the show! I'm hooked!	2015-08-24 03:07:17	True	Sneaky Pete	4
1	5	Five Stars	My Kiddos LOVE this show!!	2016-04-19 21:16:50	True	Creative Galaxy	4

The 2 columns have been joined into one column. This will ensure we get the full context of the reviewer's sentiment

In [27]:

```
# Drop columns that have no anlytical and modeling use

data = data.drop(columns=['verified_purchase','description','rating_desc','average_

data.columns
```

Out[27]:

```
Index(['rating', 'timestamp', 'movie_title', 'price', 'categories',
       'sentiment', 'reviews'],
      dtype='object')
```

In [28]:

```
# Reorder columns
columns_order = ['rating', 'timestamp', 'movie_title', 'price',
       'categories','reviews','sentiment']
data = data[columns_order]

print(data.columns)
```

```
Index(['rating', 'timestamp', 'movie_title', 'price', 'categories', 'reviews',
       'sentiment'],
      dtype='object')
```

In [29]:

```
data.head(3)
```

Out[29]:

	rating	timestamp	movie_title	price	categories	reviews	sentiment
0	5	2015-08-24 03:07:17	Sneaky Pete	14.570061	Suspense	Five Stars Amazon, please buy the show! I'm h...	positive
1	5	2016-04-19 21:16:50	Creative Galaxy	14.570061	Kids	Five Stars My Kiddos LOVE this show!!	positive
2	4	2016-09-16 18:39:49	Seven Girlfriends	12.990000	Comedy	Four Stars It was pretty good. Had some prett...	positive

Columns are reordered and will proceed to tokenize the reviews column

In [30]:
```
# Remove any html tags on the reviews text
def strip_html_tags(text):
    soup = BeautifulSoup(text, "html.parser")
    stripped_text = soup.get_text()
    return stripped_text
```

In [31]:
```
data['reviews'] = data['reviews'].apply(strip_html_tags)
data['reviews'][:3]
```

Out[31]:
```
0    Five Stars  Amazon, please buy the show! I'm h...
1                Five Stars  My Kiddos LOVE this show!!
2    Four Stars  It was pretty good. Had some prett...
Name: reviews, dtype: object
```

In [32]:
```
# Define a function for text preprocessing
def preprocess_text(text):
    # Remove numbers
    text = re.sub(r'\d+', '', text)

    # Remove punctuation and symbols
    text = re.sub(r'[^\w\s]', '', text)

    # Convert to lowercase
    text = text.lower()

    # Lemmatize text
    lemmatizer = WordNetLemmatizer()
    text = ' '.join([lemmatizer.lemmatize(word) for word in text.split()])

    # Remove stop words
    stop_words = set(stopwords.words('english'))

    text = ' '.join([word for word in text.split() if word not in stop_words])

    return text
```

In [33]:
```
# Apply the preprocessing function to the 'reviews'
data["clean_review"] = data["reviews"].apply(preprocess_text)

# verify the changes

data['clean_review'].head()
```

Out[33]:
```
0           five star amazon please buy show im hooked
1                           five star kiddos love show
2    four star wa pretty good pretty comical scene ...
3           ok love story bit slow unsatisfying ending
4    trash movie based solely incest storyline garb...
Name: clean_review, dtype: object
```

The 'reviews' column has now been transformed into lowercase strings without numbers and symbols. The next step is to tokenize the cleaned text. Tokenization is the process of converting the tweets into individual words. This facilitates better analysis of the words.

In [34]:
```
# Define a function for tokenization
def tokenize_text(text):
    # Tokenize the text
    tokens = word_tokenize(text)
    return tokens
```

In [35]:
```
# Create a new column for the tokenized text
data['reviews_tokenized'] = data['reviews'].apply(tokenize_text)

data['reviews_tokenized'].head(3)
```

Out[35]:
```
0    [Five, Stars, Amazon, ,, please, buy, the, sho...
1    [Five, Stars, My, Kiddos, LOVE, this, show, !, !]
2    [Four, Stars, It, was, pretty, good, ., Had, s...
Name: reviews_tokenized, dtype: object
```

The reviews have been tokenized and we can now use these words to perform various EDA tasks.

In [36]:
```
data.head(3)
```

Out[36]:

	rating	timestamp	movie_title	price	categories	reviews	sentiment	clean_review
0	5	2015-08-24 03:07:17	Sneaky Pete	14.570061	Suspense	Five Stars Amazon, please buy the show! I'm h...	positive	five star amazon please buy show im hooked
1	5	2016-04-19 21:16:50	Creative Galaxy	14.570061	Kids	Five Stars My Kiddos LOVE this show!!	positive	five star kiddos love show
2	4	2016-09-16 18:39:49	Seven Girlfriends	12.990000	Comedy	Four Stars It was pretty good. Had some prett...	positive	four star wa pretty good pretty comical scene ...

The reviews have been tokenized and we are now ready to perform EDA!

In [37]:
```
# save clean dataset
data.to_csv('reviews.csv', index=False)
```

2.1 Exploratory Data Analysis

In this section we will:

- **Sentiment Distribution:** Use bar charts and value counts to understand the class balance of sentiment labels (positive, negative, neutral). This helps identify any class imbalances.
- **Length Analysis:** Visualize the ditribution of review lengths (word counts) across sentiment classes. This helps identify whether positive,neutral or negative review have more (or fewer) words
- **Trend Analysis:** Show trends in sentiment distribution over time using the timestamp column.Helps identify whether positive, negative, or neutral sentiments dominate during certain timeframes.
- **Sentiment Bivariate Relationships:** Visulize the distribution of sentiments across different categories (genres),and also price (paid or free).Highlights which genres and

paid/not paid(priced zero) for content are more likely to have positive, negative, or neutral reviews.

- **Visualize Top Common Words:** Identify and display the `top 20` most common words by `sentiment` in the dataset. This gives an overview of the predominant terms.

- **Word Distributions Analysis:** Generate word clouds for positive, negative, and neutral tweets to visualize the most common `words`, `bigrams` and `trigrams` in each sentiment class. This provides a visual representation of word frequency and sentiment-specific terms.

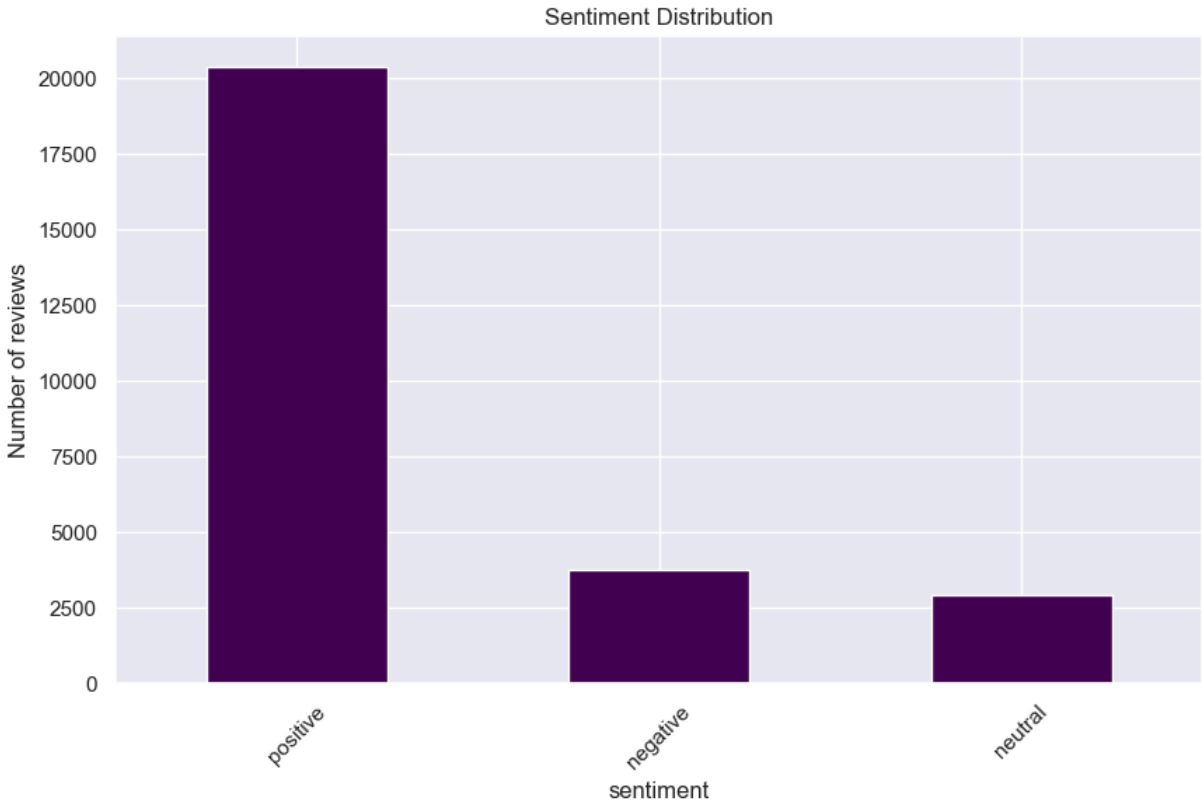
- **POS Tagging** - This helps break down text into language components such as verbs, adverbs, adjectives, nouns etc that can help further analyse language tone and distinctions between sentiments

2.1.1 Sentiment Distribution

In [38]:
```
def plot_bar_chart(df, x_label, y_label, title, stacked=False):
    """
    Plots a bar chart showing sentiment distribution for a given categorical variab

    :param df: Pandas DataFrame containing the data
    :param title: Title of the chart
    :param stacked: Boolean indicating whether bars should be stacked
    :param colormap: Color scheme for the chart
    """

    # Create figure with predefined size
    plt.figure(figsize=(10, 6))

    # Plot bar chart
    df.plot(kind="bar", stacked=stacked, colormap="viridis")

    plt.title(title)
    plt.xlabel(x_label)
    plt.ylabel(y_label)
    plt.xticks(rotation=45);
```

In [39]:
```
# Sentiment distribution bar chart
sentiment_counts = data['sentiment'].value_counts()
```

In [40]:
```
plot_bar_chart(sentiment_counts, 'sentiment', 'Number of reviews', 'Sentiment Distr
plt.savefig('sentiment_distribution')
```



- `Positive Reviews` dominate, with significantly more reviews in this category (over 20,000). This suggests a strong leaning toward positivity in the dataset.
- `Negative Reviews` are far fewer, numbering around 3,000. This indicates fewer people expressed negative sentiments.
- `Neutral Reviews` are the least frequent, with around 2,500 reviews. This suggests a smaller portion of reviews are neither strongly positive nor negative.

This class imbalance could have implications for sentiment analysis, such as potential biases in a predictive model if the imbalance isn't addressed. Addressing these imbalances (e.g., via resampling, weighting, or other methods) will be considered during modeling.

2.1.2 Length Analysis

Length analysis invoves examining the length of text data, such as the number of words, characters, or sentences, and can serve several key purposes:

- Text length can be used as a feature in machine learning models. For example, longer texts might convey more nuanced sentiments, while shorter ones could tend to be more polarized or direct.
- Patterns in text length can reveal insights, such as whether positive or negative sentiments are associated with longer explanations or brief remarks.

In this section we will analyze both `character lengths` and `word counts`

Length of Characters Analysis

In [41]:
```
# check for the length (characters) of a review
data['reviews'][0]
```

Out[41]:
```
"Five Stars  Amazon, please buy the show! I'm hooked!"
```

In [42]:
```
length_sample = len(data['reviews'][0])
print(f'Length of a sample review: {length_sample}')
```

```
Length of a sample review: 52
```

In [43]:
```
data['Length'] = data['reviews'].str.len()
data.head(3)
```

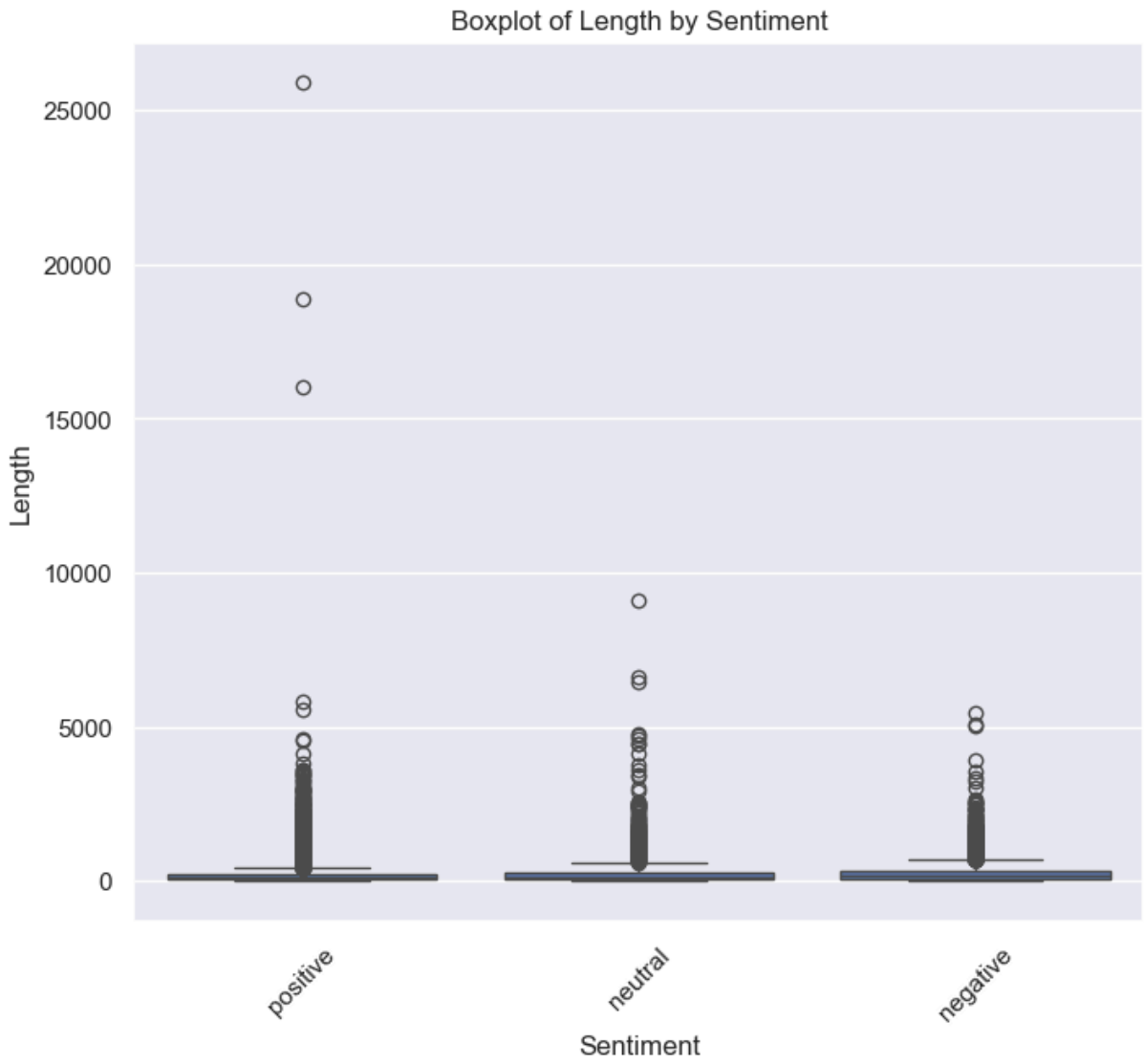
Out[43]:

	rating	timestamp	movie_title	price	categories	reviews	sentiment	clean_review
0	5	2015-08-24 03:07:17	Sneaky Pete	14.570061	Suspense	Five Stars Amazon, please buy the show! I'm h...	positive	five star amazon please buy show im hooked
1	5	2016-04-19 21:16:50	Creative Galaxy	14.570061	Kids	Five Stars My Kiddos LOVE this show!!	positive	five star kiddos love show
2	4	2016-09-16 18:39:49	Seven Girlfriends	12.990000	Comedy	Four Stars It was pretty good. Had some prett...	positive	four star wa pretty good pretty comical scene ...

In [44]:
```
# create a function for box plots
def box_plot(df, col):
    fig, ax = plt.subplots(figsize=(8,7))
    sns.boxplot(x="sentiment", y=col, data=df)
    plt.title(f"Boxplot of {col} by Sentiment")
    plt.xlabel("Sentiment")
    plt.ylabel(col)
    plt.xticks(rotation=45);
```

In [45]:
```
# plot the review length by sentiment
box_plot(data, 'Length')
```

```
plt.savefig('length_by_sentiment')
```



Positive Sentiment

- Wider distribution with reviews ranging from `concise` to `extremely detailed`. This reflects that users with positive sentiments often express themselves in more elaborate ways, but there's also variability with shorter feedback included.
- The presence of outliers with higher character counts suggests that some users go into detail when praising the product or content.

Neutral Sentiment

- Neutral reviews exhibit a slightly wider distribution compared to negative reviews, with word counts that fall between `concise` and `moderately descriptive`.
- The lack of extreme outliers and the `middle-range distribution` suggest that users providing neutral feedback often limit themselves to `balanced and succinct descriptions`.

Negative Sentiment

- Most concise reviews, with a `narrow distribution and lower median counts`. This suggests that users expressing negative feedback tend to keep their reviews `brief and to the point`.
- The narrow range implies consistency in word count, as users often convey dissatisfaction in a few impactful words.

Word Count Analysis

- This entails number of words in a review

Advantages of word counts include:

1. Identifying Review Quality

- Short reviews (1-5 words) → Often vague ("Good", "Bad", "Works well")
- Medium-length reviews (10-50 words) → More informative but still concise
- Long reviews (50+ words) → Typically more detailed.

2. Enhancing Sentiment Analysis

- Short reviews might have stronger polarity ("Terrible!" = negative, "Amazing!" = positive).
- Longer reviews may contain mixed sentiment or more detail about the movie.

In [46]:
```
# word count
word_count = data['reviews'][0].split()
print(f'Word count in a sample review: {len(word_count)}')
```

```
Word count in a sample review: 9
```

In [47]:
```
def word_count(review):
    review_list = review.split()
    return len(review_list)
```

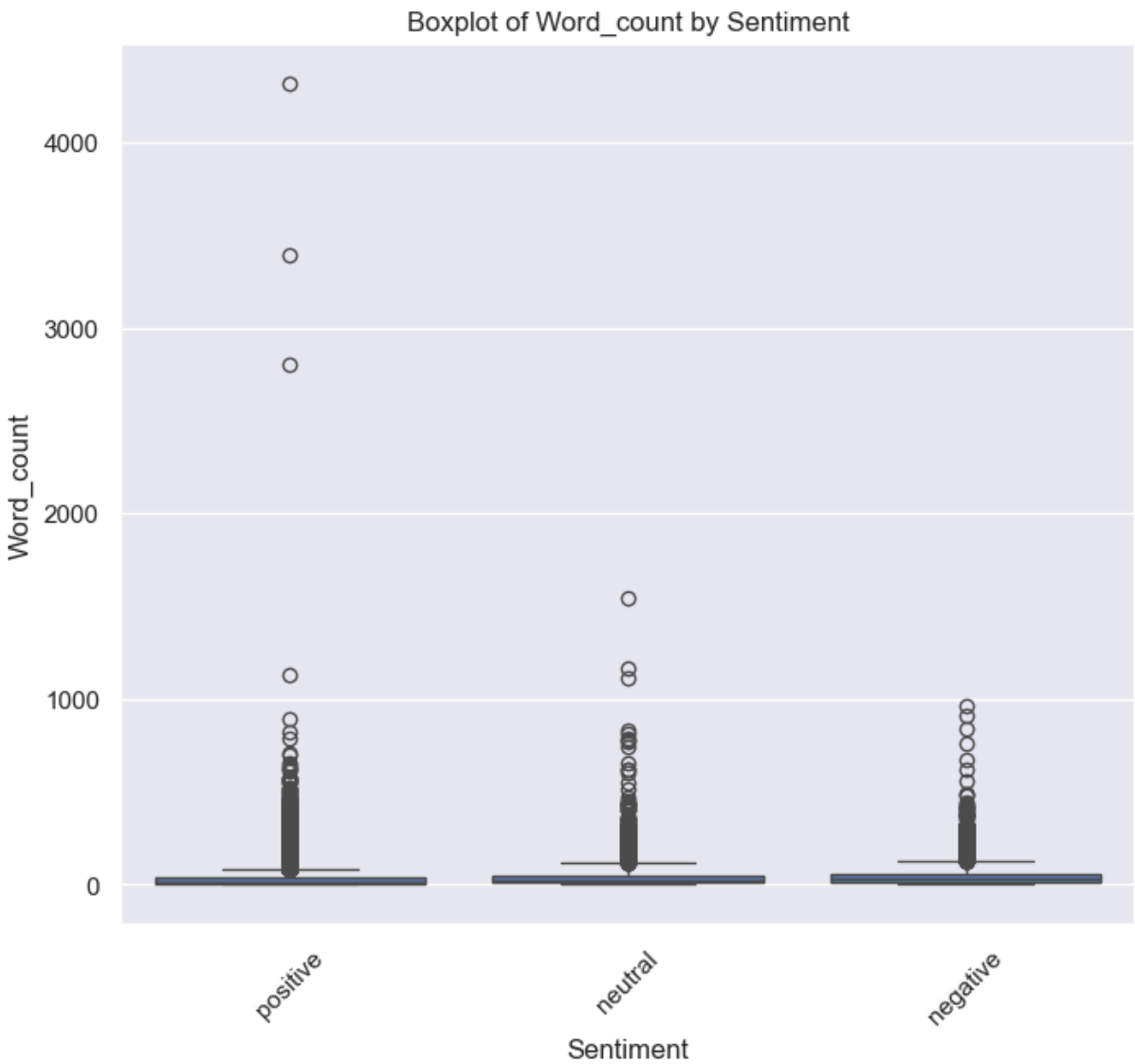
In [48]:
```
data['Word_count'] = data['reviews'].apply(word_count)
data.head(3)
```

Out[48]:

	rating	timestamp	movie_title	price	categories	reviews	sentiment	clean_review
0	5	2015-08-24 03:07:17	Sneaky Pete	14.570061	Suspense	Five Stars Amazon, please buy the show! I'm h...	positive	five star amazon please buy show im hooked
1	5	2016-04-19 21:16:50	Creative Galaxy	14.570061	Kids	Five Stars My Kiddos LOVE this show!!	positive	five star kiddos love show
2	4	2016-09-16 18:39:49	Seven Girlfriends	12.990000	Comedy	Four Stars It was pretty good. Had some prett...	positive	four star wa pretty good pretty comical scene ...

In [49]:

```
# Plot the word count by sentiment
box_plot(data, 'Word_count')
plt.savefig('word_count_by_sentiment')
```



We are seeing a similar pattern as in the character length (not surprising) because more words,more characters.

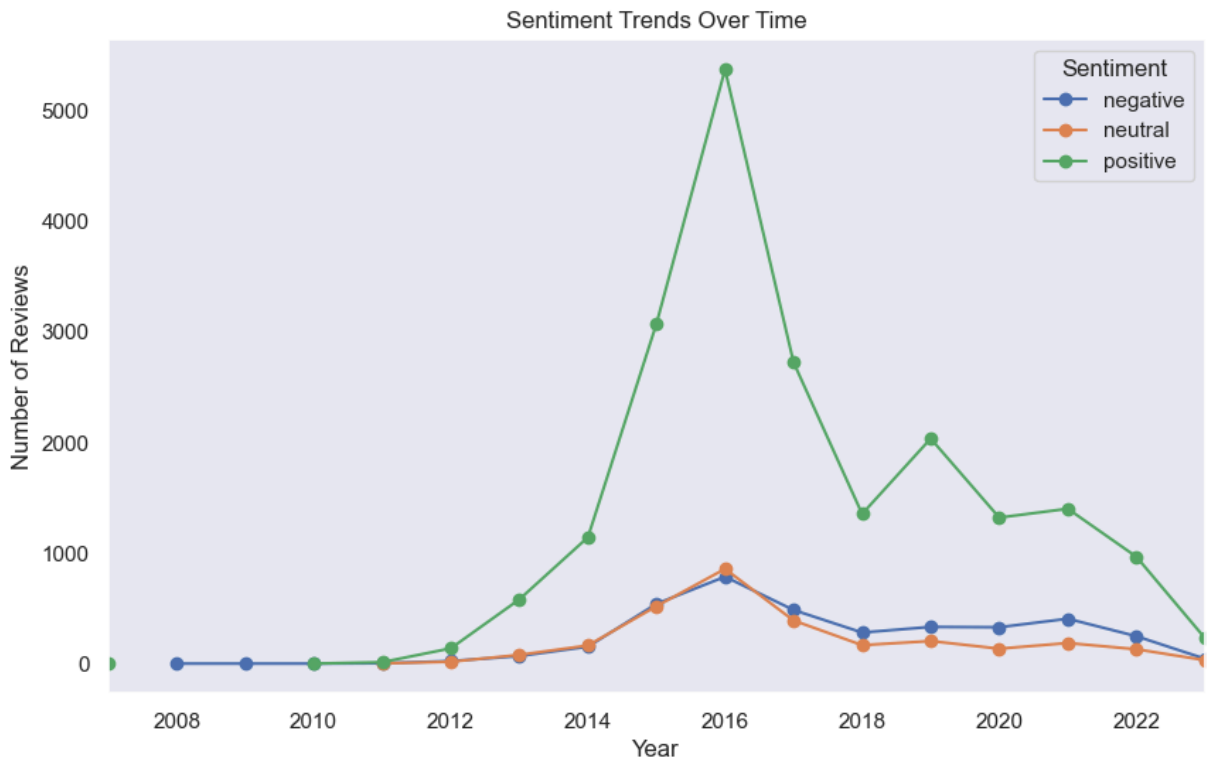
- The `positive' class shows more disparity in word lengths,implying that positive reviews have moreoutliers with extremely wordy reviews.
- The `neutral` and `negative` classes indicate a tighter conventration on the number of words with a higher disparity in the `neutral` class.

It is clear that most `negative` reviews are succinct and to the point.

2.1.3 Trend Analysis

In [50]:
```
# Group by Year and sentiment
time_sentiment = data.groupby([data['timestamp'].dt.to_period('Y'), 'sentiment']).s

# Plot sentiment trends over time
time_sentiment.plot(kind='line', figsize=(10, 6), marker='o')
plt.title('Sentiment Trends Over Time')
```

```
plt.xlabel('Year')
plt.ylabel('Number of Reviews')
plt.legend(title='Sentiment')
plt.grid()
plt.savefig('sentiments_over_time')
plt.show()
```



Positive Sentiment (Green Line)

- Dominates over the entire time period, peaking dramatically around 2016 with over 5,000 reviews. This surge could reflect an event, product, or content that received overwhelming positive attention during that year.
- After 2016, positive reviews show a decline, leveling off around 2,500–3,000 reviews by 2022.

Negative Sentiment (Blue Line)

- Remains relatively stable with low counts throughout the years, consistently staying below 1,500 reviews.
- A slight increase can be observed around 2016 and 2018, possibly corresponding to specific content or events that triggered more negative feedback.

Neutral Sentiment (Orange Line)

- Similar to negative sentiments, neutral reviews are consistently low, hovering around 1,000 or fewer reviews throughout the years.
- There is a small increase around 2016, though less pronounced than the spike in positive sentiments.

Overall Patterns

- The dramatic peak in positive sentiment around 2016 suggests a significant trend or popularity surge. Negative and neutral sentiments, while increasing slightly at the same time, remain far lower.
- After 2016, all sentiments gradually decline, though positive reviews still outnumber the others by a wide margin.

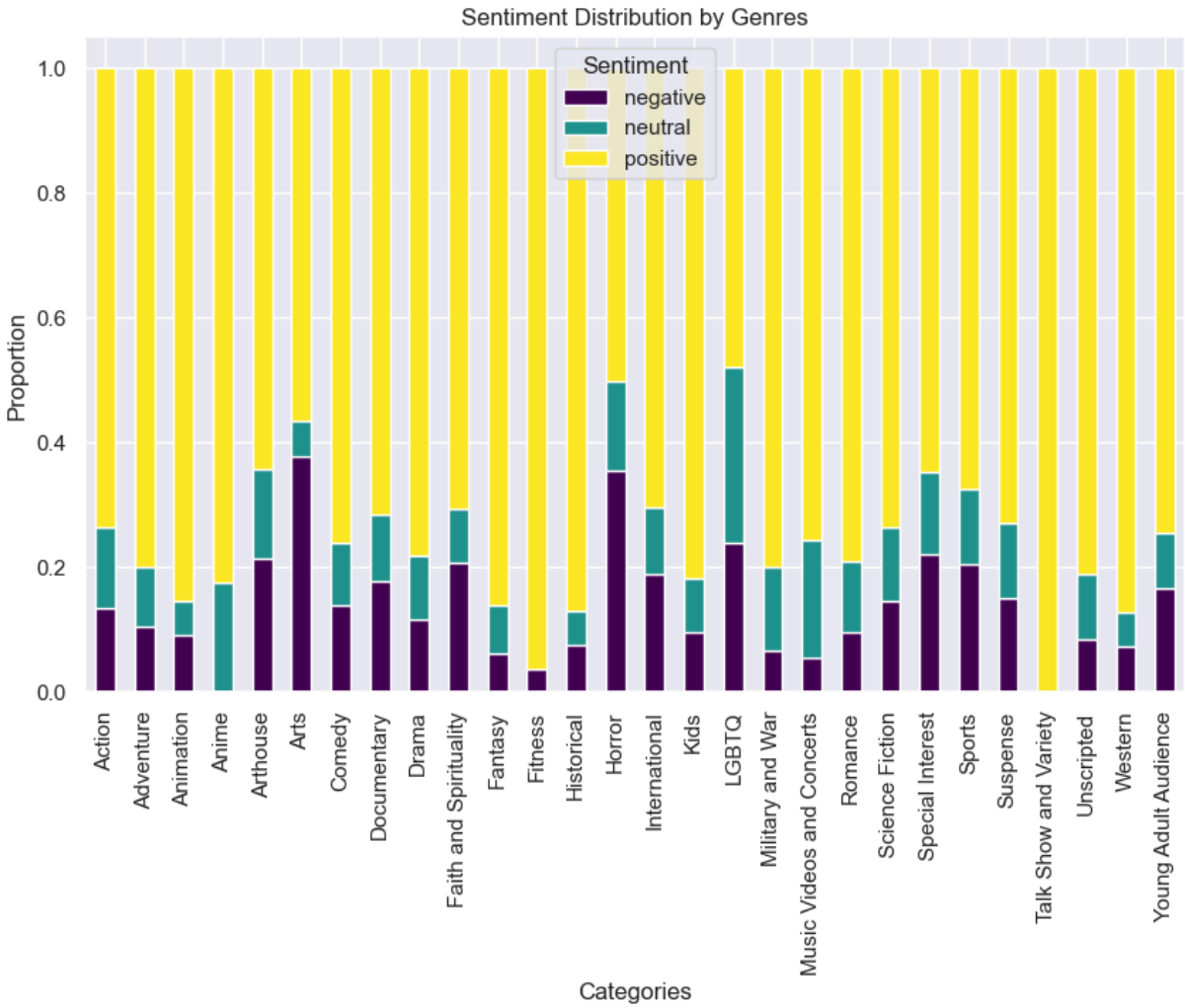
Possible Next Steps

- Investigate what happened around 2016 that drove the massive increase in positive reviews.
- Explore factors contributing to the relatively stable presence of negative and neutral sentiments compared to the fluctuating positive trend.

2.1.4 Sentiment Bivariate Analysis

2.1.4.1 Sentiment by Genres (Categories)

In [51]:
```
# Cross-tabulation for sentiment and categories
category_sentiment = pd.crosstab(data['categories'], data['sentiment'], normalize='

# Stacked bar chart
category_sentiment.plot(kind='bar', stacked=True, figsize=(10, 6), colormap='viridi
plt.title('Sentiment Distribution by Genres')
plt.xlabel('Categories')
plt.ylabel('Proportion')
plt.legend(title='Sentiment')
plt.savefig('sentiment_by_genre')
plt.show()
```



Positive Sentiment (Yellow)

- Categories like `Talk Show and Variety` and `fitness` have the highest proportion of positive sentiments, indicating a generally favorable reception in these genres. It could also be because these genres usually have a `niche` as opposed to a more `general` audience
- `Animation`, `Fantasy`, `Historical` and `Western` and also show strong positive sentiments, likely reflecting their appeal to general audiences.

Neutral Sentiment (Teal)

- Categories like `Anime`, `LGBTQ`, `Music Videos and Concerts` have notable proportions of neutral sentiments. This suggests these genres may elicit more informational or balanced reviews, rather than strong emotional reactions.
- The `Arthouse` and `Action` categories also display significant neutral sentiments.

Negative Sentiment (Purple)

- Categories `Arts`, `Horror`, `LBTQ`,stand out with higher proportions of negative sentiment. This may indicate dissatisfaction with certain content or unmet expectations.

Key Insights

- Audience Preferences: Categories like `Animation`, `Kids`,' `Fantasy`, `Fitness`, `Historical`, `Talkshow` and `Western` are highly favored, while genres such as `Horror` and `Arts` often elicit critical feedback.
- Balanced Reception: Categories with higher neutral sentiment, such as `LGBTQ` and `Music` content, suggest these genres may attract more nuanced reactions.

Categories with higher negative sentiment could benefit from analyzing specific user feedback to address shortcomings.

2.1.4.1 Sentiment by Price (Paid or Free)

We would like to analyze whether sentiments are better, worse or neutral depending on whether the content is paid or free.

In [52]:
```
# Create a binary column for paid/free
data['is_free'] = data['price'].apply(lambda x: 'Free' if x == 0 else 'Paid')

# Cross-tabulation for sentiment and price
price_sentiment = pd.crosstab(data['is_free'], data['sentiment'], normalize='index'
```

In [53]:
```
plot_bar_chart(price_sentiment, 'Price Category', 'proportion', 'Sentiment Distribu
plt.savefig('sentiment_by_price')
```

```
<Figure size 1000x600 with 0 Axes>
```

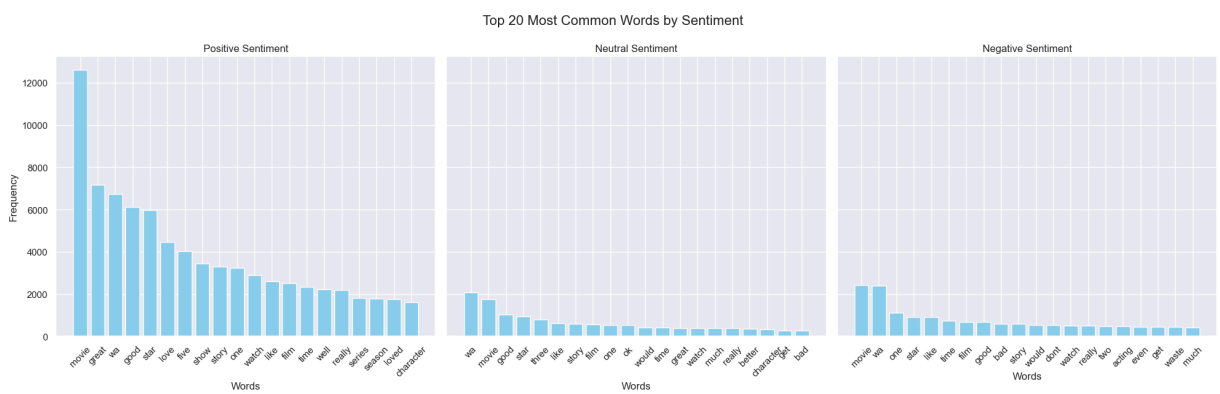
- Both free and paid content have the highest proportion of positive sentiment, showing that users generally express more favorable opinions, regardless of whether the content was free or paid.
- Negative sentiment appears slightly more in free content, possibly reflecting dissatisfaction with expectations when the content is free or of lower quality.

2.1.5 Top 20 Common Words

In this section, we want to visualize the distinctios or similarities between the top words by sentiment. A clearer distinction of words between sentiments is an indication that the model has better decision boundaries between the classes.

In [54]:
```
# Function to get top common words for a given sentiment
def get_top_words(sentiment):
    sentiment_reviews = ' '.join(data[data['sentiment'] == sentiment]['clean_review
    return Counter(sentiment_reviews).most_common(20)

# Get common words for each sentiment
positive_words = get_top_words('positive')
neutral_words = get_top_words('neutral')
negative_words = get_top_words('negative')
```

```
# Prepare data for plotting
positive_words, positive_counts = zip(*positive_words)
neutral_words, neutral_counts = zip(*neutral_words)
negative_words, negative_counts = zip(*negative_words)

# Create subplots for each sentiment
fig, axes = plt.subplots(1, 3, figsize=(20, 6), sharey=True)

# Plot for Positive Sentiment
axes[0].bar(positive_words, positive_counts, color='skyblue')
axes[0].set_title('Positive Sentiment')
axes[0].set_xlabel('Words')
axes[0].set_ylabel('Frequency')
axes[0].tick_params(axis='x', rotation=45)

# Plot for Neutral Sentiment
axes[1].bar(neutral_words, neutral_counts, color='skyblue')
axes[1].set_title('Neutral Sentiment')
axes[1].set_xlabel('Words')
axes[1].tick_params(axis='x', rotation=45)

# Plot for Negative Sentiment
axes[2].bar(negative_words, negative_counts, color='skyblue')
axes[2].set_title('Negative Sentiment')
axes[2].set_xlabel('Words')
axes[2].tick_params(axis='x', rotation=45)

# Adjust layout and show the plot
plt.tight_layout()
plt.suptitle('Top 20 Most Common Words by Sentiment', fontsize=16, y=1.05)
plt.savefig('top_20_words_by_sentiment')
plt.show()
```



Positive Sentiment

- Words like `movie`, `great`, `good`, `love`, `show`, and `story` dominate.These words convey enthusiasm and strong approval, which aligns with a positive sentiment.
- Words such as `time`, `really`, `loved`, and `well` suggest detailed appreciation or personal connections to the content.
- Words like `series` and `season`, `character` might indicate users favorably reviewing TV shows or multi-part content.

Positive sentiment reviews are often more specific, expressing satisfaction, emotional attachment, and appreciation for the quality of the movie or series.

Neutral Sentiment

- Words such as `movie`, `good`, `story`, `film`, `ok`, `much` and `would` are prevalent.
- The word `ok` stands out, reflecting the nature of neutral feedback where users are neither overly impressed nor dissatisfied.
- Words like `better` and `character` indicate neutral feedback may involve `constructive criticism`, suggesting areas of improvement.
- `Great` appearing alongside `ok` suggests ambivalence in sentiment reviews, where both positive and mediocre tones exist.

Neutral sentiment reviews have mixed descriptions, often balancing praise with critique or indifference.This may present a problemlater on where the model struggles to distinguish between the positive and negative classes

Negative Sentiment

- Words like `movie`, `bad`, `don't`, `waste`, `acting`, `even`, `time` and `get` dominate.
- Negative expressions such as `bad` and `waste` clearly express dissatisfaction.
- Words like `acting` and `even` suggest users criticize specific aspects like `performance` or the `quality of content`.
- The word `don't` appearing frequently implies negative reviews often contain direct discouragement or warnings.

Negative sentiment reviews are concise and direct, focusing on dissatisfaction or unmet expectations, with fewer descriptive words compared to positive reviews.

Overall Insight

- Words like `movie` and `time` are frequent across all sentiment classes, showing the consistency of these topics regardless of sentiment.
- Positive reviews tend to use descriptive and emotionally charged words, while neutral reviews balance opinion, and negative reviews are focused on direct criticism or dissatisfaction.

2.1.6 Word Clouds

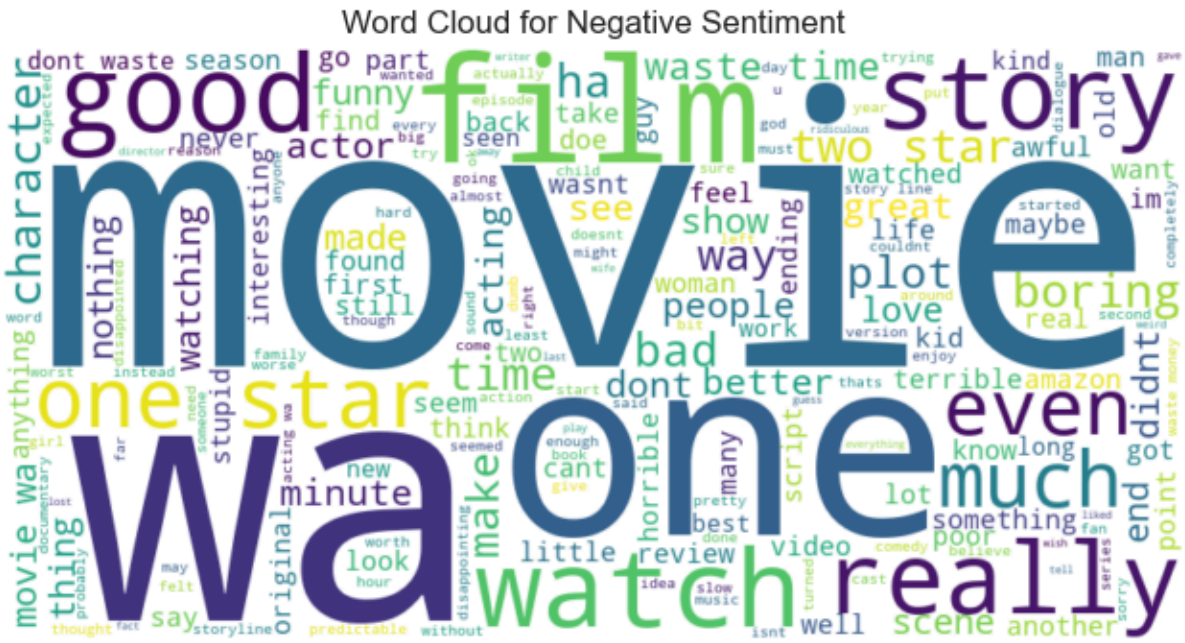
This is aslo another way to visulailize the didtribution of words across sentments with the frequency of the word indicated by the font

In [55]:
```
# Generate word clouds by sentiment
for sentiment in data['sentiment'].unique():
    sentiment_text = ' '.join(data[data['sentiment'] == sentiment]['clean_review'])
    wordcloud = WordCloud(width=800, height=400, background_color='white').generate
```

```
# Plot word cloud
plt.figure(figsize=(8, 4))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title(f'Word Cloud for {sentiment.capitalize()} Sentiment')
plt.savefig('word_clouds_by_sentiment')
plt.show()
```

Word Cloud for Positive Sentiment

five star
great movie
four star
watch
one
wa
film
good movie
character
really
show
story
time
see
think
great
star
still
season
interesting
life
fun
much
good
ha
way
make
entertaining
serie
movie
wa
even
love
people
new
little
made
lot
classic
best
enjoy
episode
love movie
movie five
actor
world
many
funny
loved
well done
star good
never
part
plot
work
watching
original
amazing
find
look
say
first
story line
always
thing
real
bad
know
wonderful
doe
family
comedy
action
amazon
love
beautiful
excellent
great show
movie love
love show
take
thought
different
year old
enjoyable
play
performance
worth watching
watched
found
didnt
great series
enough
nice
music
may
true
drama
tv
friend
excellent movie
star excellent
without
nothing
guy
keep
loved movie
one best
hope
sure
got
old
go
documentary
review
wish
wife
cast
age
enjoyed movie
one favorite
really enjoyed
kind
around
course
video
probably
feel
bit
sweet
awesome
storyline
need
come
though
right
must see
im
great acting
seem
want
another
hilarious
comedy
sad
thats
amazon
love love
good story
especially
definitely
nice
back
everything liked
pretty good
going
ending
fan
history
end
u
actually
scene
role
quite
humor
overall
everyone
book
give
year
child
day
two
really good
almost
great story
cant wait
humor
everyone
book
give
year
child
day
kid
star love
watching
enjoyed movie
performance
original
work



- Positive sentiment reviews heavily focus on praising storytelling, character development, and overall emotional engagement. The frequent mention of `great`, `love`, `good` and `five-star` highlights the satisfaction and excitement users associate with their favorite content.
- Neutral sentiments seem to provide balanced evaluations, often describing technical or storyline aspects of movies or TV shows. The frequent mention of words tied to storytelling - `character`, `story` shows that neutral reviewers are more focused on describing, rather than judging, the content.
- Negative sentiments words like `boring`, `waste`, `awful`, and `bad` reflect a severe level of dissatisfaction and directly convey negative emotion.

2.1.7 Top N-grams

N-grams, including trigrams (3-grams), are critical in Natural Language Processing (NLP) as they help capture context and relationships between words in text. Here's why they matter:

`Context Preservation`: N-grams consider sequences of words, allowing models to understand dependencies and improve predictions.

`Improved Features`: They enhance text representations by capturing multi-word phrases (e.g., `artificial intelligence`), which are often more meaningful than individual words.

`Sentiment and Semantic Analysis`: Trigrams provide richer features for detecting subtleties, such as negations or idiomatic expressions (e.g., `not very good`).

Bigrams

In [56]:
```
# Create a function to get n-grams
def get_top_ngrams(text, ngram_range=(2,2), n=10):
    vectorizer = CountVectorizer(ngram_range=ngram_range, stop_words="english")
    ngram_counts = vectorizer.fit_transform(text)
    ngram_sum = np.asarray(ngram_counts.sum(axis=0)).flatten()

    ngrams_freq = list(zip(vectorizer.get_feature_names_out(), ngram_sum))
    ngrams_freq = sorted(ngrams_freq, key=lambda x: x[1], reverse=True)[:n]

    return ngrams_freq
```

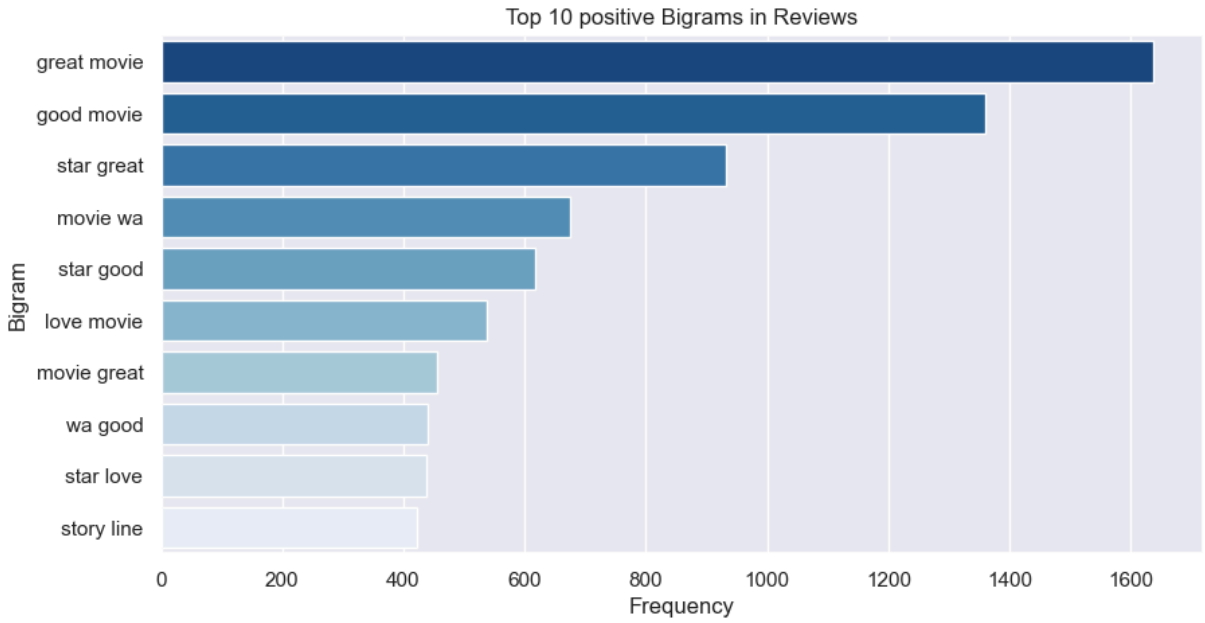
In [57]:
```
# Filter reviews by sentiment
positive_reviews = data[data["sentiment"] == "positive"]["clean_review"]
neutral_reviews = data[data["sentiment"] == "neutral"]["clean_review"]
negative_reviews = data[data["sentiment"] == "negative"]["clean_review"]
```

In [58]:
```
# Get top bigrams
top_positive_bigrams = get_top_ngrams(positive_reviews, ngram_range=(2, 2), n=10)
top_neutral_bigrams = get_top_ngrams(neutral_reviews, ngram_range=(2, 2), n=10)
top_negative_bigrams = get_top_ngrams(negative_reviews, ngram_range=(2, 2), n=10)
```

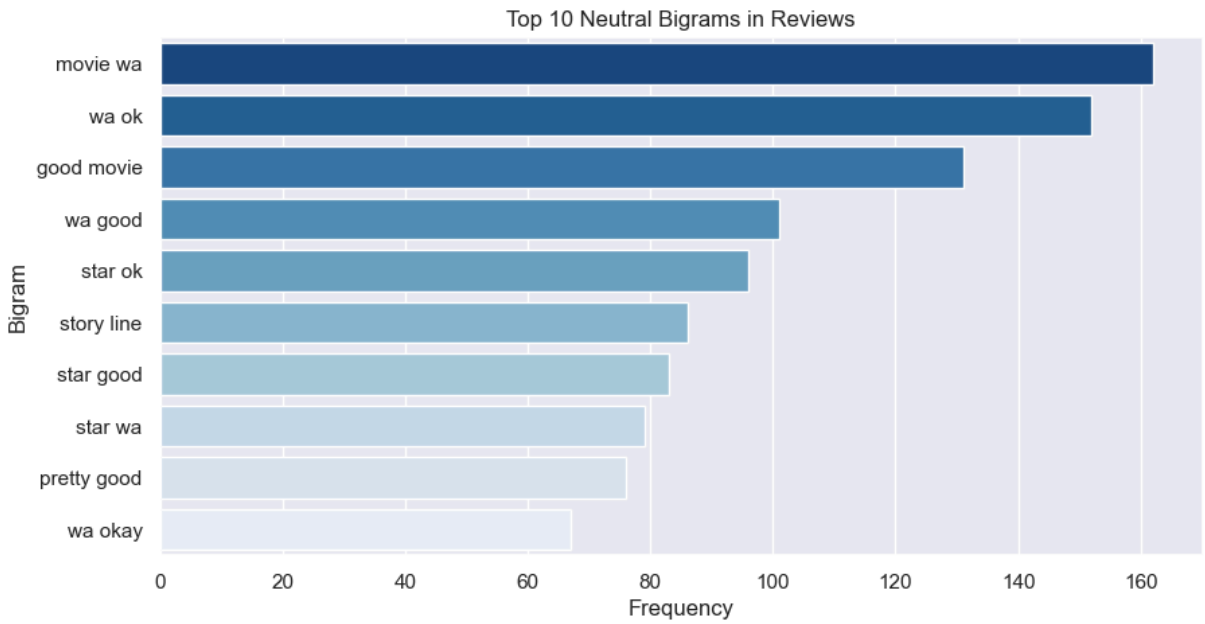
In [59]:
```
# positive Bigrams dataframe
positive_bigram_df = pd.DataFrame(top_positive_bigrams, columns=["Bigram", "Frequen

# neutral bigrams df
neutral_bigram_df = pd.DataFrame(top_neutral_bigrams, columns=["Bigram", "Frequency

# negative bigrams df
negative_bigram_df = pd.DataFrame(top_negative_bigrams, columns=["Bigram", "Frequen
```

In [60]:
```
def plot_ngram_frequency(ngram_df, x_col="Frequency", y_col="Bigram", title="Top N-
    """
    Plots a bar chart of the most frequent n-grams.

    Parameters:
        ngram_df (DataFrame): A DataFrame containing n-grams and their frequencies.
        x_col (str, optional): The column name for frequency (default: "Frequency")
        y_col (str, optional): The column name for n-grams (default: "Bigram").
        title (str, optional): Title of the plot (default: "Top N-grams in Reviews"
        palette (str, optional): Color palette for the bar chart (default: "Blues_r
    """
    plt.figure(figsize=(10, 5))
    sns.barplot(y=y_col, x=x_col, data=ngram_df, palette=palette)
    plt.title(title)
    plt.xlabel(x_col)
    plt.ylabel(y_col);
```

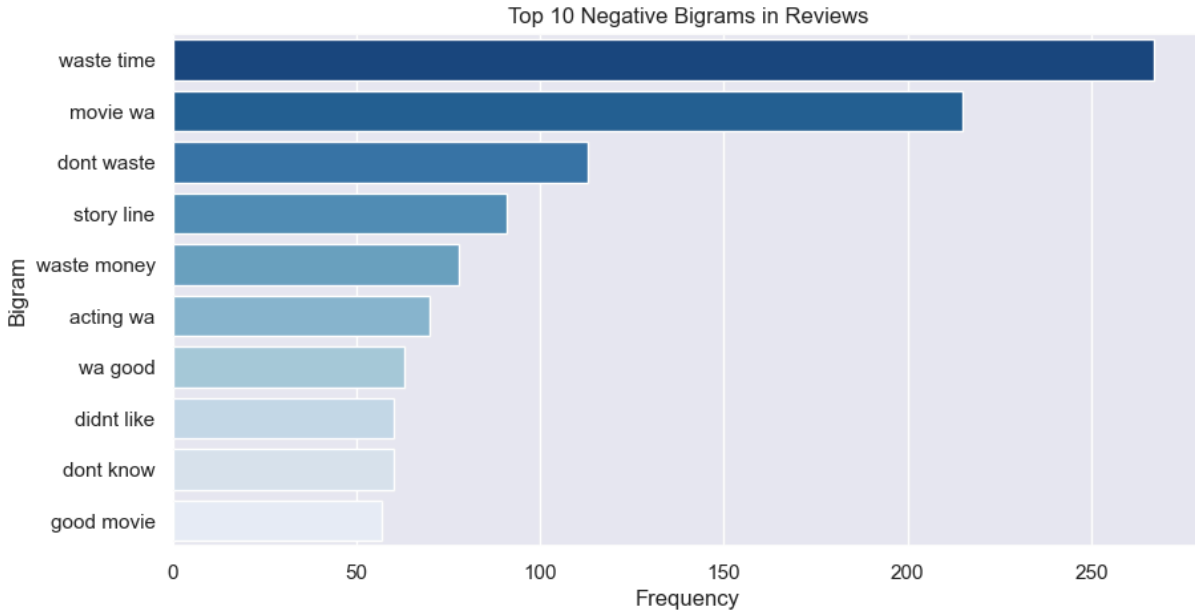
In [61]:
```
plot_ngram_frequency(positive_bigram_df, title="Top 10 positive Bigrams in Reviews"
plt.savefig('top_10_positive_bigrams')
```



In [62]:
```
plot_ngram_frequency(neutral_bigram_df, title="Top 10 Neutral Bigrams in Reviews")
plt.savefig('top_10_neutral_bigrams')
```



In [63]:
```
plot_ngram_frequency(negative_bigram_df, title="Top 10 Negative Bigrams in Reviews"
plt.savefig('top_10_negative_bigrams')
```



`Positive` : Bigram charts reflect uplifting phrases like `great movie` and `love it`, showcasing satisfaction and enthusiasm.

`Neutral` : Reviews use reserved language such as `wa ok` and `story line` focusing on straightforward descriptions and mild evaluations.

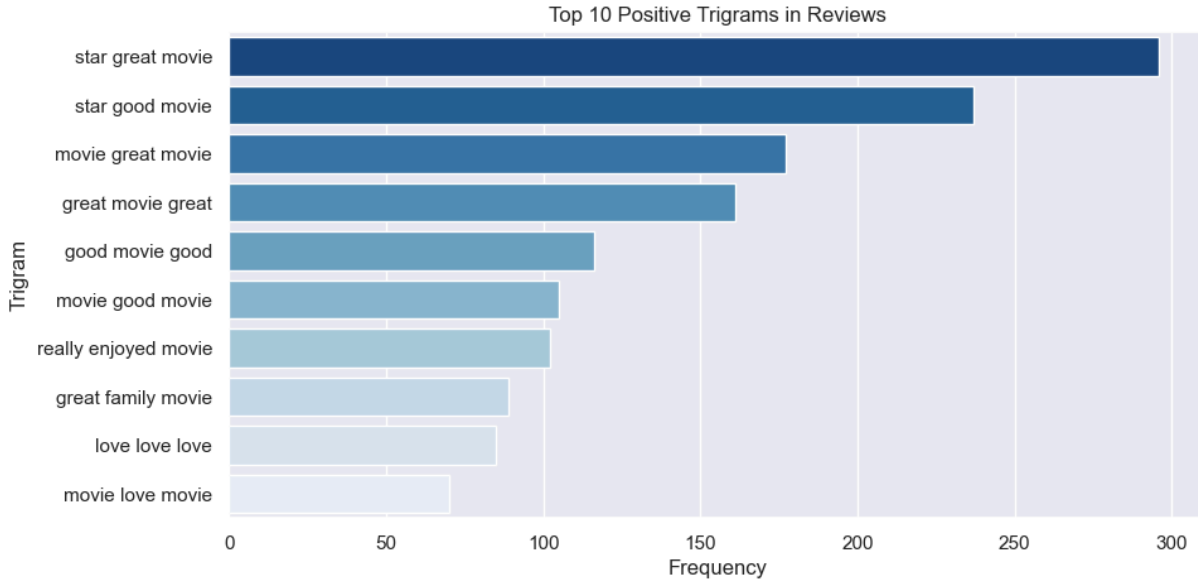
`Negative` : Strong dissatisfaction emerges with terms like `waste time` and `waste money` reflecting frustration and disappointment.

Tri-grams

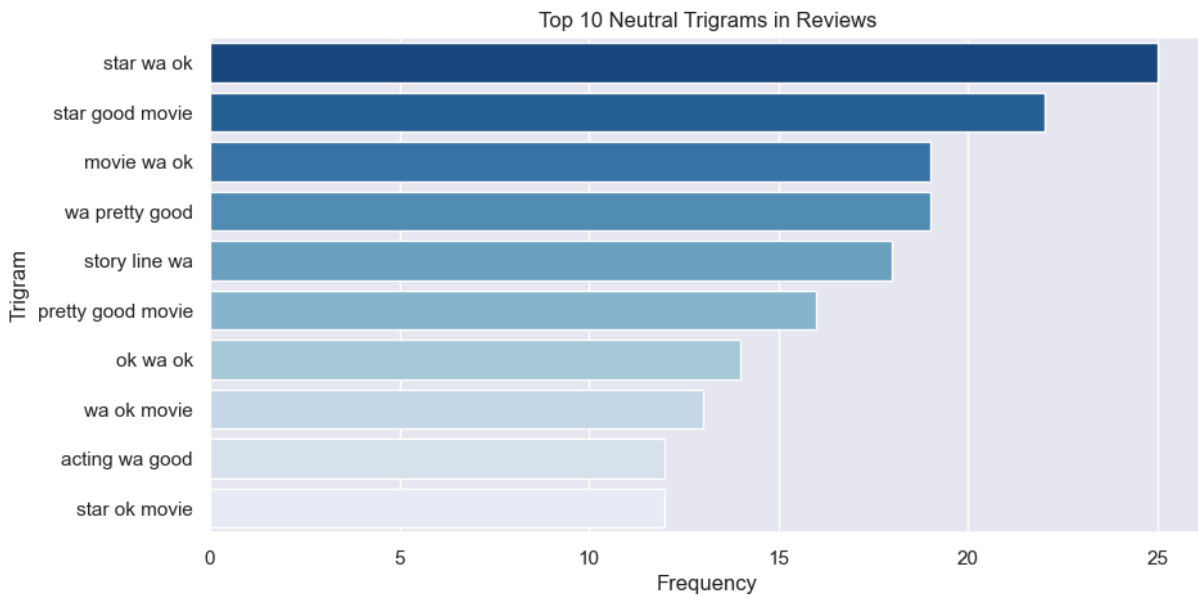
In [64]:
```
# Get top trigrams
top_positive_trigrams = get_top_ngrams(positive_reviews, ngram_range=(3, 3), n=10)
top_neutral_trigrams = get_top_ngrams(neutral_reviews, ngram_range=(3, 3), n=10)
top_negative_trigrams = get_top_ngrams(negative_reviews, ngram_range=(3, 3), n=10)
```

In [65]:
```
# Positive Trigrams DataFrame
positive_trigram_df = pd.DataFrame(top_positive_trigrams, columns=["Trigram", "Freq

# Neutral Trigrams DataFrame
neutral_trigram_df = pd.DataFrame(top_neutral_trigrams, columns=["Trigram", "Freque

# Negative Trigrams DataFrame
negative_trigram_df = pd.DataFrame(top_negative_trigrams, columns=["Trigram", "Freq
```

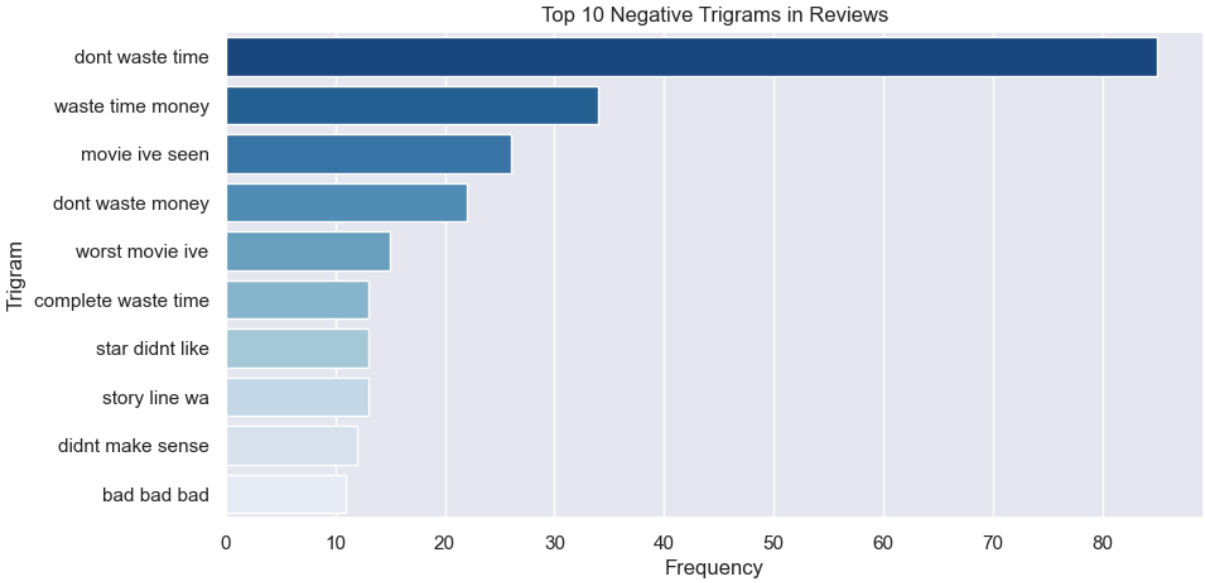
In [66]:
```
plot_ngram_frequency(positive_trigram_df, y_col="Trigram", title="Top 10 Positive T
plt.savefig('top_10_positive_trigrams')
```



In [67]:
```
plot_ngram_frequency(neutral_trigram_df, y_col="Trigram", title="Top 10 Neutral Tri
plt.savefig('top_10_neutral_bigrams')
```



In [68]:
```
plot_ngram_frequency(negative_trigram_df, y_col="Trigram", title="Top 10 Negative T
plt.savefig('top_10_negative_bigrams')
```



Positive Reviews

- `star great movie` and `star good movie`: These trigrams suggest that viewers often associate a well-loved star (actor or celebrity) with a movie they consider great or good. Star power seems to heavily influence positive sentiment
- `movie great movie` and `movie good movie`: These could reflect general appreciation for the movie as a whole, showing that simple, enthusiastic expressions dominate positive feedback.
- `great movie great` and `good movie good`: These repetitive trigrams emphasize a strong, direct liking for the movie, showcasing excitement and reinforcement.
- `love love love` and `movie love movie`: These phrases show deep enthusiasm or passion for a movie. Such expressions are highly emotional and indicate strong viewer attachment.

Neutral Reviews

- `star wa ok`: Suggests moderate satisfaction with the performance of a star or actor.
- `wa pretty good`: Expresses mild appreciation, implying that the content was enjoyable but not exceptional.
- `story line wa`: Suggests viewers found the storyline worth mentioning, though the sentiment remains neutral.
- `acting wa good`: Highlights adequate or competent performances by the cast, without strong enthusiasm.

Negative Reviews

- `dont waste time`: Indicates strong dissatisfaction and regret over spending time on the content.
- `worst movie ive`: Expresses extreme displeasure, labeling the movie as one of the worst they've seen.

- `story line wa` : Suggests negative feedback about the storyline or plot quality.
- `bad bad bad` : Demonstrates outright disapproval with strong emotional negativity.

2.1.8 Perform POS Tagging

- We'll extract the most common `ADVERBS` and `ADJECTIVES` to analyze the language used in reviews.This is useful in sentiment analysis as improves text classification. We will focus on Adverbs and Adjectives as these often carry sentiment.

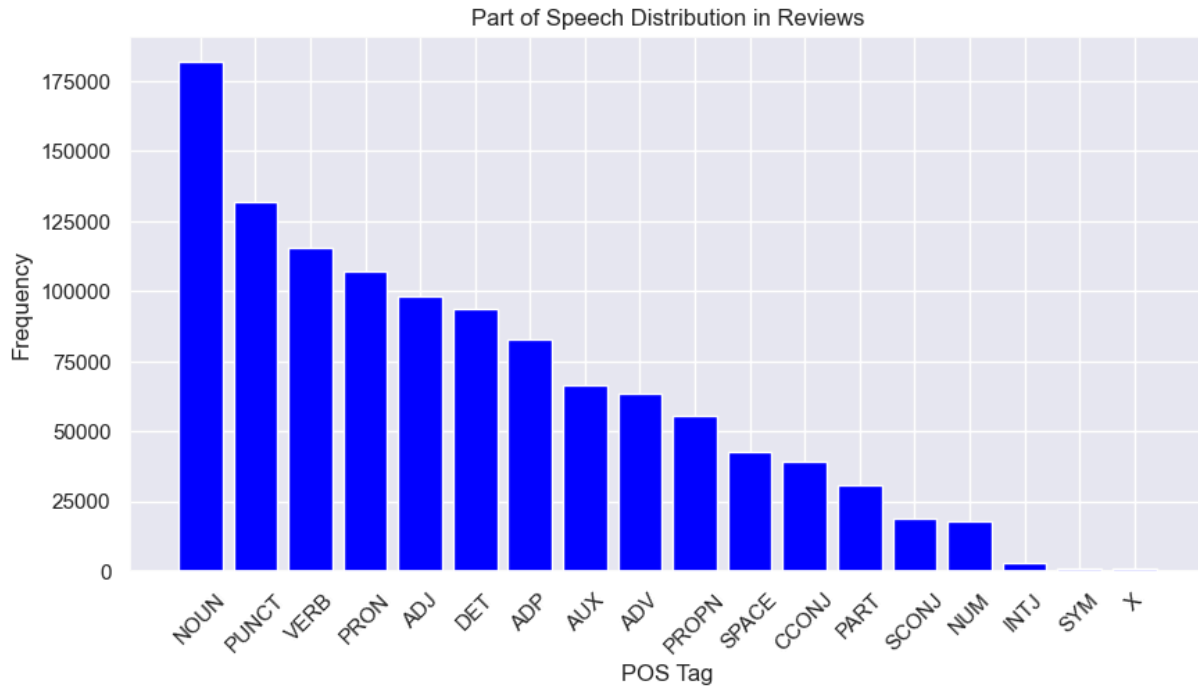
In [69]:
```
# Load the English NLP model from spaCy
nlp = spacy.load("en_core_web_sm")
```

In [70]:
```
def pos_tagging(text):
    doc = nlp(text)
    pos_counts = Counter(token.pos_ for token in doc)
    return pos_counts
```

In [71]:
```
# Apply POS tagging to all reviews
data["pos_counts"] = data["reviews"].apply(pos_tagging)
```

In [72]:
```
# Aggregate POS frequencies
pos_totals = Counter()
for pos_dict in data["pos_counts"]:
    pos_totals.update(pos_dict)

# Convert to DataFrame
pos_df = pd.DataFrame(pos_totals.items(), columns=["POS", "Count"])
pos_df = pos_df.sort_values(by="Count", ascending=False)
```

In [73]:
```
# Plot POS distribution
plt.figure(figsize=(10, 5))
plt.bar(pos_df["POS"], pos_df["Count"], color="blue")
plt.title("Part of Speech Distribution in Reviews")
plt.xlabel("POS Tag")
plt.ylabel("Frequency")
plt.xticks(rotation=45)
plt.savefig('POS_distribution')
plt.show();
```



Extract Most Common Adverbs & Adjectives

In [74]:
```
def extract_words(text, pos_type):
    doc = nlp(text)
    words = [token.text.lower() for token in doc if token.pos_ == pos_type and not
    return words
```

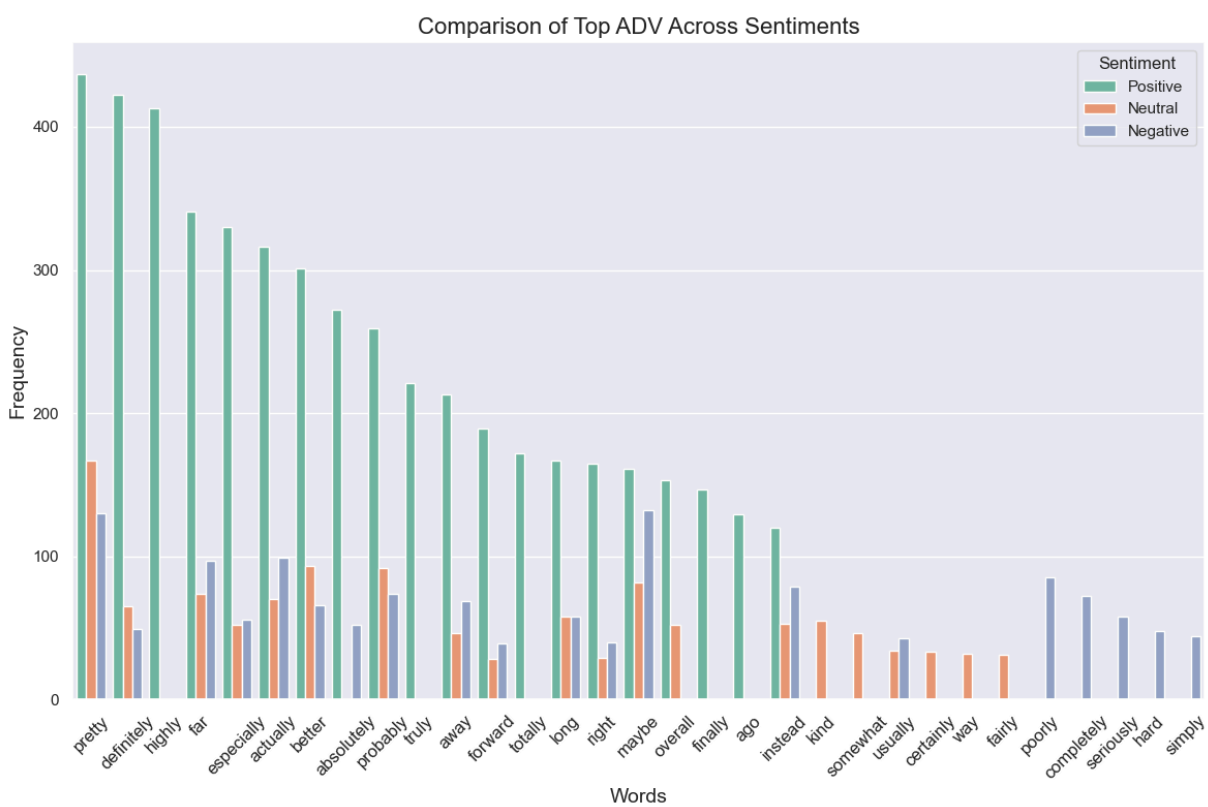
In [75]:
```
# Function to extract POS words for a specific sentiment
def get_pos_by_sentiment(data, sentiment_label, pos_type):
    filtered_texts = data[data["sentiment"] == sentiment_label]["reviews"]
    words = []
    filtered_texts.apply(lambda x: words.extend(extract_words(x, pos_type)))
    return Counter(words).most_common(20)  # Return top 20 most common words
```

In [76]:
```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

def plot_pos_words_comparison(data_dict, pos_type, figsize=(12, 8), color_palette="
    """
    Plots a grouped bar chart to compare POS word frequencies across sentiments.

    Args:
    - data_dict (dict): Dictionary where keys are sentiment labels (e.g., "positive
      and values are lists of (word, count) from the Counter object.
    - pos_type (str): The type of POS tags analyzed (e.g., "ADJ" or "VERB").
    - figsize (tuple): Size of the figure (default: (12, 8)).
    - color_palette (str): Color scheme for the chart (default: "Set2").
    """
    # Flatten the data into a DataFrame for plotting
    data = []
    for sentiment_label, words_counts in data_dict.items():
        for word, count in words_counts:
```

```
            data.append({"Word": word, "Frequency": count, "Sentiment": sentiment_l

    df = pd.DataFrame(data)

    # Create grouped bar chart
    plt.figure(figsize=figsize)
    sns.barplot(data=df, x="Word", y="Frequency", hue="Sentiment", palette=color_pa
    plt.xlabel('Words', fontsize=14)
    plt.ylabel('Frequency', fontsize=14)
    plt.title(f'Comparison of Top {pos_type} Across Sentiments', fontsize=16)
    plt.xticks(rotation=45, fontsize=12)
    plt.legend(title="Sentiment")
    plt.tight_layout()  # Adjust layout to avoid clipping
    plt.show()
```

In [77]:
```
# Extract words for each sentiment
positive_words = get_pos_by_sentiment(data, "positive", "ADV")
neutral_words = get_pos_by_sentiment(data, "neutral", "ADV")
negative_words = get_pos_by_sentiment(data, "negative", "ADV")

# Combine into a dictionary
data_dict = {
    "Positive": positive_words,
    "Neutral": neutral_words,
    "Negative": negative_words
}
```

In [78]:
```
plot_pos_words_comparison(data_dict, pos_type="ADV")
plt.savefig('top_adverbs_by_sentiment')
```




```
<Figure size 640x480 with 0 Axes>
```

Positive Sentiment:

- Adverbs such as `pretty`, `definitely`, `highly`, `better` and `absolutely` are predominantly used in positive reviews, as shown by tall green bars. These adverbs convey `certainty`, `intensity`, and `emphasis`, reinforcing positive expressions.
- Words like `totally` and `truly` further highlight strong emotional engagement in favorable sentiments.

Neutral Sentiment:

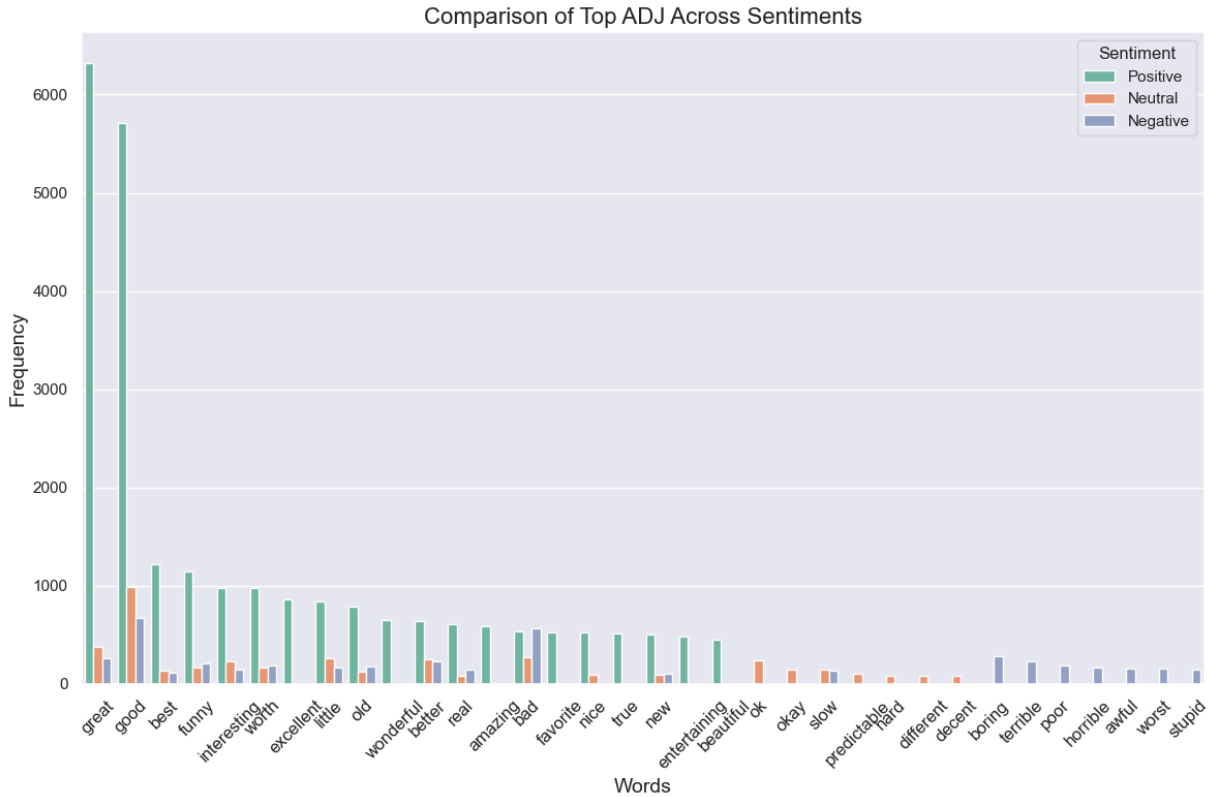
- Adverbs like `maybe`, `overall` and `actually` appear balanced, often used to convey reserved or factual observations. The presence of these words suggests a more measured tone in neutral contexts.
- Adverbs such as `usually` and `fairly` also show moderate usage, reflecting habitual or mild evaluations.

Negative Sentiment:

- Adverbs like `poorly`, `hard` and `seriously` are significant in negative reviews, with the blue bars showing notable frequencies. These words express frustration or dissatisfaction.
- Adverbs such as `ago` and `simply` often reflect critical reflections or blunt assessments of an event or experience.

In [79]:
```
# Extract words for each sentiment
positive_words = get_pos_by_sentiment(data, "positive", "ADJ")
neutral_words = get_pos_by_sentiment(data, "neutral", "ADJ")
negative_words = get_pos_by_sentiment(data, "negative", "ADJ")

# Combine into a dictionary
data_dict = {
    "Positive": positive_words,
    "Neutral": neutral_words,
    "Negative": negative_words
}
```

In [80]:
```
plot_pos_words_comparison(data_dict, pos_type="ADJ")
plt.savefig('top_adjectives_by_sentiment')
```




```
<Figure size 640x480 with 0 Axes>
```

Positive Sentiment:

- Adjectives like `great`, `wonderful` and `excellent` dominate this category, reflecting highly favorable opinions. These terms frequently appear in positive reviews, indicating strong enthusiasm and satisfaction.
- The high frequency of these adjectives suggests that positive reviewers tend to emphasize superlative qualities.

Neutral Sentiment:

- Adjectives such as `okay` and `predictable` are moderately frequent in neutral reviews. These adjectives often carry a descriptive tone but lack strong emotional intensity.
- The presence of neutral adjectives reflects a measured or objective stance, offering straightforward evaluations.

Negative Sentiment:

- Adjectives like `horrible`, `terrible` and `boring` are prominent in this category. They highlight dissatisfaction and critique, especially regarding performance or quality.
- The frequency of these adjectives indicates frustration or disappointment as key themes in negative reviews.

This analysis demonstrates how adjectives differ in intensity and emotional weight across sentiments, providing critical insight for tasks like sentiment classification.

3.0 Modeling

In this section:

1. We will split the split the dataset into train, validation and test sets.
2. Train 3 baseline models and evaluate the results using classification report and confusion matrix.
3. Hyperparameter tune the best baseline model and evaluate the results
4. Use advanced modeling techniques like LSTM and BERT Transformations to further improve the model performance

3.1 Split the dataset into train, validation and test sets

The training set will be used to fit the model and learn patterns from the data; we will use the **validation set** to tune hyperparameters (learning rate, number of layers etc) and evaluate the model during training. The validation set allows the comparison of different models and configurations to choose the best performing, without leaking information to the test set. We will use the **test set** to provide an unbiased evaluation of the final model's performance, mimicking how it would perform on real-world data.

- We will stratify the dataset to ensure classdistribution in each split remains similar to the orignal dataset.
- Test Size is set at 20%
- Validation set is set at 25% of the remaining 80% ,resulting in a finalsplit of 60% training,20% validation and 20% test.
- Random state ensures reprodicibility so that we can get the same splits each time.

In [81]:
```
# Define feature and target columns
X = data['clean_review']  # Features
y = data['sentiment']  # Target

# Split the dataset into training + validation and test sets
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

# Further split the training + validation set into training and validation sets
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, stratify=y_temp, random_state=42
)

# Print sizes of the sets
print("Training set size:", len(X_train))
print("Validation set size:", len(X_val))
print("Test set size:", len(X_test))
```

```
Training set size: 16190
Validation set size: 5397
Test set size: 5397
```

3.2 Baseline Models

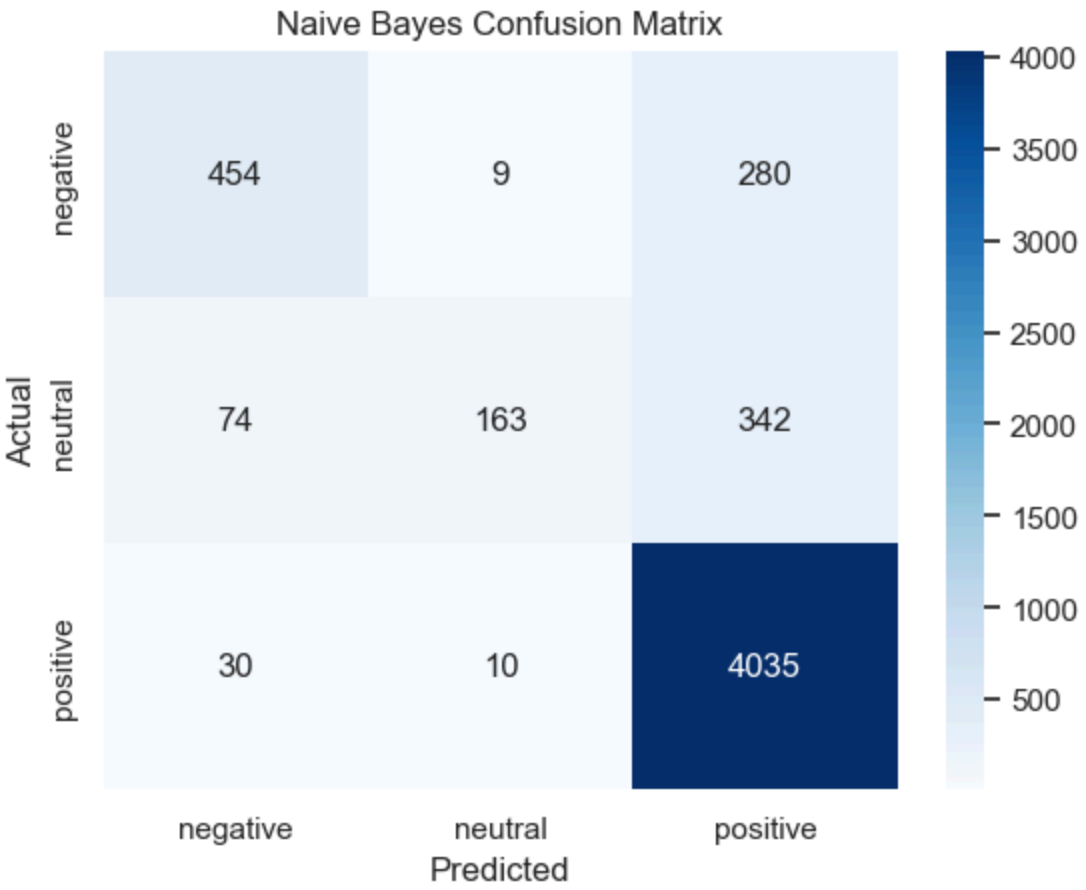
In this section, we will train and evaluate a Multinomial Naive Bayes (NB), Support Vector Machine (SVM), and Random Forest (RF) using the TF-IDF Vectorizer.

The TF-IDF Vectorizer's parameters `max_features` (to limit the number of features used in the dataset) and `ngram_range` (to includes unigram, bigrams and trigrams for better context understanding) will be adjusted.

We will evaluate the best baseline model and further tune the hyperparameters to improve on the performance.

In [82]:
```
# Naive Bayes Pipeline
nb_pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=5000,ngram_range=(1,3))),# Include bigra
    ('nb', MultinomialNB())  # MultinomialNB inherently handles class imbalance wit
])

# Fit the pipeline on training data
nb_pipeline.fit(X_train, y_train)

# Predict on validation data
y_val_pred_nb = nb_pipeline.predict(X_val)

# Classification report and confusion matrix for NB
nb_dict = classification_report(y_val, y_val_pred_nb, output_dict=True)
nb_df = pd.DataFrame(nb_dict).transpose()
print("NB Classification Report:\n",round(nb_df,2))

nb_cm = confusion_matrix(y_val, y_val_pred_nb)
sns.heatmap(nb_cm, annot=True, fmt='d', cmap='Blues', xticklabels=nb_pipeline.class
plt.title("Naive Bayes Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.savefig("Confusion Matrix Baseline NB")
plt.show()
```

```
NB Classification Report:
               precision  recall  f1-score  support
negative            0.81    0.61      0.70   743.00
neutral             0.90    0.28      0.43   579.00
positive            0.87    0.99      0.92  4075.00
accuracy            0.86    0.86      0.86     0.86
macro avg           0.86    0.63      0.68  5397.00
weighted avg        0.86    0.86      0.84  5397.00
```



Positive Class

- The model performed exceptionally well for the positive class, predicting `4035` out of `4075` samples correctly.
- However, this high accuracy may be due to the imbalance in the dataset, where the positive class overwhelms the others.

Negative Class

- `280` out of `743` negative samples are misclassified as positive and a smaller amount `9` as neutral.

Neutral Class

- many instances are misclassified as positive `342`, suggesting the model is biased toward predicting the majority class and/or distinguishing between the neutral and the positive class. This is not surprising as the top word analysis as well as POS tagging show that the wordings used in the neutral review, while reserved, tends towards positivity. e.g. `ok`, `good` etc

Class Imbalance Impact

The classifier shows a strong bias for the positive class, likely due to its dominance in the training data. The minority classes (negative and neutral) suffer from high misclassification

rates as a result. Despite an accuracy of `86%`,the recall for the negative and neutral classes are sub-optimal at `61% and 28%` respectively.

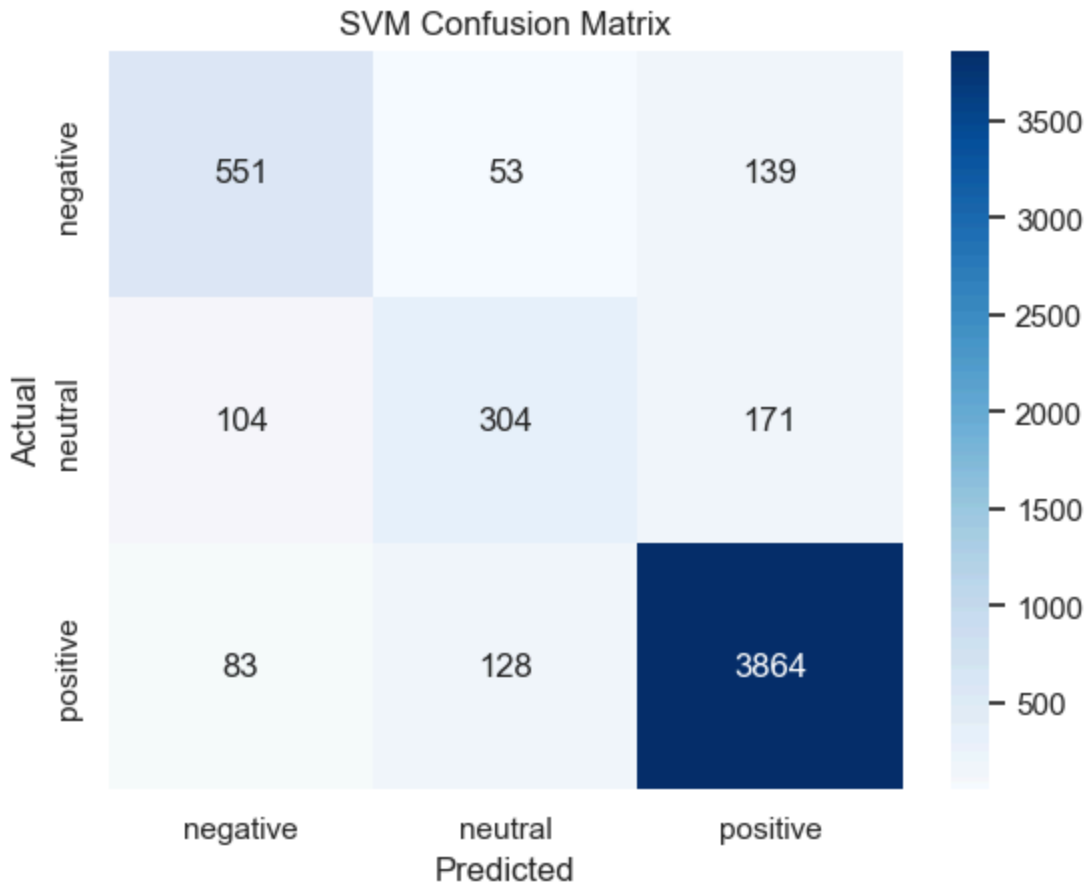
Next Steps Implement SVM and Random Forest models that allow weighting of classes

3.2.2 Baseline SVM Model

class_weight = 'balanced' assigns weights to classes inversely proportional to their frequency,compensating for imbalance

In [83]:
```
# Support Vector Machine Pipeline
svm_pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=5000,ngram_range=(1,3))),
    ('svm', SVC(class_weight='balanced', random_state=42)) # 'class_weight=balanced
])

# Fit the svm pipeline
svm_pipeline.fit(X_train, y_train)

# Predict on validation data
y_val_pred_svm = svm_pipeline.predict(X_val)

# Classification report and confusion matrix for SVM
svm_dict = classification_report(y_val, y_val_pred_svm, output_dict=True)
svm_df = pd.DataFrame(svm_dict).transpose()
print("SVM Classification Report:\n",round(svm_df,2))


svm_cm = confusion_matrix(y_val, y_val_pred_svm)
sns.heatmap(svm_cm, annot=True, fmt='d', cmap='Blues', xticklabels=svm_pipeline.cla
plt.title("SVM Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.savefig("Confusion Matrix Baseline SVM")
plt.show()
```

```
SVM Classification Report:
               precision  recall  f1-score  support
negative            0.75    0.74      0.74   743.00
neutral             0.63    0.53      0.57   579.00
positive            0.93    0.95      0.94  4075.00
accuracy            0.87    0.87      0.87     0.87
macro avg           0.77    0.74      0.75  5397.00
weighted avg        0.87    0.87      0.87  5397.00
```



Strong Positive Class Performance

- The model correctly predicted `3864` a `Recall of 95%` out of the total positive samples, demonstrating strong performance for the positive sentiment class.
- A few positive samples were misclassified as neutral (128) or negative (83).

Moderate Performance for the Negative Class

- 551 out of the negative class samples were predicted correctly - `Recall of 74%`
- There were `139` cases misclassified as positive, which might be concerning, as these errors could significantly affect interpretation as we are looking to correctly identify negative reviews.
- A smaller portion `53 samples` was misclassified as neutral.

Challenges with the Neutral Class

- This class saw significant confusion, with only `304 samples` correctly classified.
- A large number of neutral samples were misclassified as positive `178` or negative `83`.
- This is a common issue in sentiment analysis, as neutral classes often overlap with positive and negative sentiments in text.

Recommendation for improvement

- Try an enemble model like Random Forest

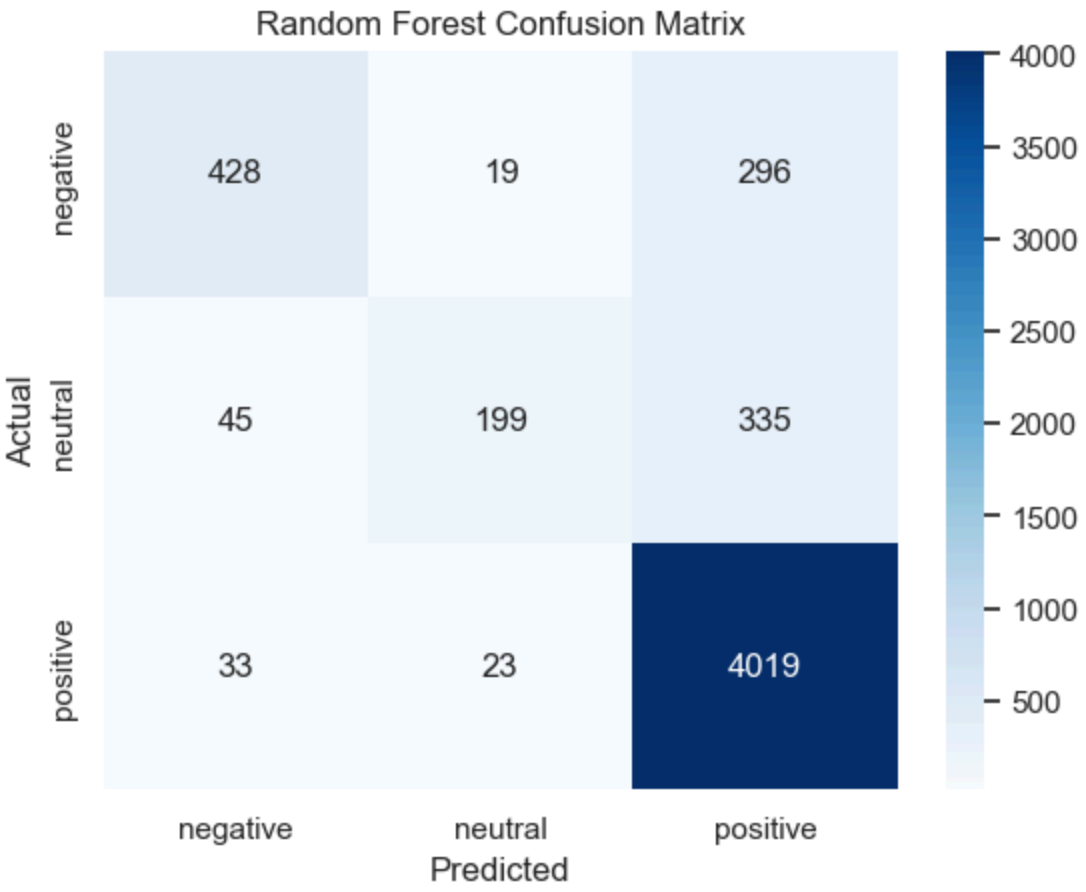
- Hyperparameter tuning - e.g.kernel type `linear`, `rbf`), regularization parameters `C` or `gamma` for better boundary definitions
- `Undersample` the majority class to balance the dataset

3.2.3 Random Forest Baseline Model

class_weight='balanced': Ensures that the minority class is not ignored during tree-building by giving it more weight in the loss calculation.

In [84]:
```
# Random Forest Pipeline
rf_pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=5000,ngram_range=(1,3))),
    ('rf', RandomForestClassifier(class_weight='balanced', random_state=42))  # 'cl
])


# Fit the rf pipeline
rf_pipeline.fit(X_train, y_train)

# Predict on validation data
y_val_pred_rf = rf_pipeline.predict(X_val)

# Classification report and confusion matrix for RF
rf_dict = classification_report(y_val, y_val_pred_rf, output_dict=True)
rf_df = pd.DataFrame(rf_dict).transpose()
print("RF Classification Report:\n",round(rf_df,2))

rf_cm = confusion_matrix(y_val, y_val_pred_rf)
sns.heatmap(rf_cm, annot=True, fmt='d', cmap='Blues', xticklabels=rf_pipeline.class
plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.savefig("Confusion Matrix Baseline RF")
plt.show()
```

```
RF Classification Report:
               precision  recall  f1-score  support
negative            0.85    0.58      0.69   743.00
neutral             0.83    0.34      0.49   579.00
positive            0.86    0.99      0.92  4075.00
accuracy            0.86    0.86      0.86     0.86
macro avg           0.85    0.64      0.70  5397.00
weighted avg        0.86    0.86      0.84  5397.00
```



Positive Class

- The model performs exceptionally well for the positive class, predicting `4019` out of `4075` positive instances correctly.
- `33` positive samples were misclassified as negative, and `23` as neutral, though these misclassifications are relatively low.

Negative Class

- `296` samples were misclassified as positive, which may stem from the class imbalance and overlapping features between negative and positive sentiments. '19' samples were incorrectly classified as neutral.

Neutral Class

- The most significant issue is the `335` neutral instances misclassified as positive, reflecting a potential bias toward predicting the dominant positive class. `45` neutral instances were incorrectly classified as negative.

A clear bias is evident in favor of the positive class due to its dominance in the dataset, leading to frequent misclassification of minority classes. This is evident in the `low recall (0.58, 0.34)` and `f1 scores(0.69,0.49)` of the neutral and negative classes - an indication that the model is struggling to identify a significant number of samples correctly

3.2.4 Comparing the results of the 3 baseline models and way forward

In [85]:
```
# Add a column to each DataFrame identifying the model
nb_df['Model'] = 'Naive Bayes'
svm_df['Model'] = 'SVM'
rf_df['Model'] = 'Random Forest'

# Combine the DataFrames
combined_report = pd.concat([nb_df, svm_df, rf_df], axis=0)

# Reset the index for a cleaner look
# Add a column to each DataFrame identifying the model
nb_df['Model'] = 'Naive Bayes'
svm_df['Model'] = 'SVM'
rf_df['Model'] = 'Random Forest'

# Combine the DataFrames
combined_report = pd.concat([nb_df, svm_df, rf_df], axis=0)

# Reset the index for a cleaner look
combined_report.reset_index(inplace=True)

# Pivot the combined report for a side-by-side comparison
pivot_report = combined_report.pivot(index='index', columns='Model', values=['preci

# Display the pivoted DataFrame
pivot_report = pivot_report.round(3)
pivot_report
```

Out[85]:

	precision			recall			f1-score		
Model	**Naive Bayes**	**Random Forest**	**SVM**	**Naive Bayes**	**Random Forest**	**SVM**	**Naive Bayes**	**Random Forest**	**SVM**
index									
accuracy	0.862	0.861	0.874	0.862	0.861	0.874	0.862	0.861	0.874
macro avg	0.859	0.845	0.766	0.628	0.635	0.738	0.683	0.697	0.751
negative	0.814	0.846	0.747	0.611	0.576	0.742	0.698	0.685	0.744
neutral	0.896	0.826	0.627	0.282	0.344	0.525	0.428	0.485	0.571
positive	0.866	0.864	0.926	0.990	0.986	0.948	0.924	0.921	0.937
weighted avg	0.862	0.858	0.869	0.862	0.861	0.874	0.840	0.842	0.871

`Accuracy` Naive Bayes: 86.2%; Random Forest: 86.1%; SVM: 87.4% (`highest accuracy`)

SVM achieves the highest overall accuracy, making it the best model for correctly predicting sentiments across the dataset. However, accuracy alone can be misleading due to the dataset's imbalance, so other metrics need to be considered

`Negative Sentiment` Precision: Naive Bayes: 0.81 ; Random Forest: 0.85 (`highest precision`) ; SVM: 0.75

Recall: Naive Bayes: 0.61; Random Forest: 0.58 (`lowest recall`); SVM: 0.74

F1-Score: Naive Bayes: 0.70; Random Forest: 0.69; SVM: 0.74 (`highest f1 score`)

SVM performs the best in terms of F1-score for negative sentiment, balancing both precision and recall. However, Random Forest achieves the highest precision, which may reduce false positives for the negative class

`Neutral Sentiment` Precision: Naive Bayes: 0.90(`highest precision`): Random Forest: 0.83; SVM: 0.63

Recall:Naive Bayes: 0.28; Random Forest: 0.34; SVM: 0.53 (`highest recall`)

F1-Score: Naive Bayes: 0.43; Random Forest: 0.49; SVM: 0.57 (`highest f1 score`)

Neutral sentiment is a challenging class for all models, but SVM performs slightly better in terms of recall and F1-score, correctly identifying more neutral instances. Naive Bayes has high precision but struggles significantly with recall, likely due to misclassification as other sentiments

`Positive Sentiment` Precision: Naive Bayes: 0.87; Random Forest: 0.86; SVM: 0.93

Recall: Naive Bayes: 0.99; Random Forest: 0.99; SVM: 0.95

F1-Score: Naive Bayes: 0.95; Random Forest: 0.92; SVM: 0.94

All three models perform well for the positive class, with SVM achieving the highest precision and competitive recall and F1-score. Naive Bayes and Random Forest are slightly behind but still robust for this majority class

`Weighted Average` Precision: Naive Bayes: 0.86; Random Forest: 0.86;SVM: 0.87

Recall:Naive Bayes: 0.86; Random Forest: 0.86; SVM: 0.87

F1-Score: Naive Bayes: 0.84; Random Forest: 0.84; SVM: 0.87

SVM outperforms the others in weighted averages across all metrics. This indicates that SVM's performance is skewed toward the majority class (positive), but it still handles the dataset better overall than the other two models

Conclusion Best Model Overall: SVM achieves the highest accuracy, weighted averages, and strong positive sentiment performance. However, its macro-averaged recall and F1-score

show it still struggles with minority classes.

Neutral Sentiment Weakness: All models need improvement in handling neutral sentiment, with SVM showing the most promise due to higher recall.

Next Steps Tune the SVM Model to try and improve performance especially the recall on the Neutral and Negative Classes

3.3 Hyperparameter Tuned SVM Model

Using GridSearchCV, exprole a variety of parameter combinations to optimize class separation and address the bias towards the positive class

The tuning was done in another model and the best params were identified. We are using the best params below to save on computational time.

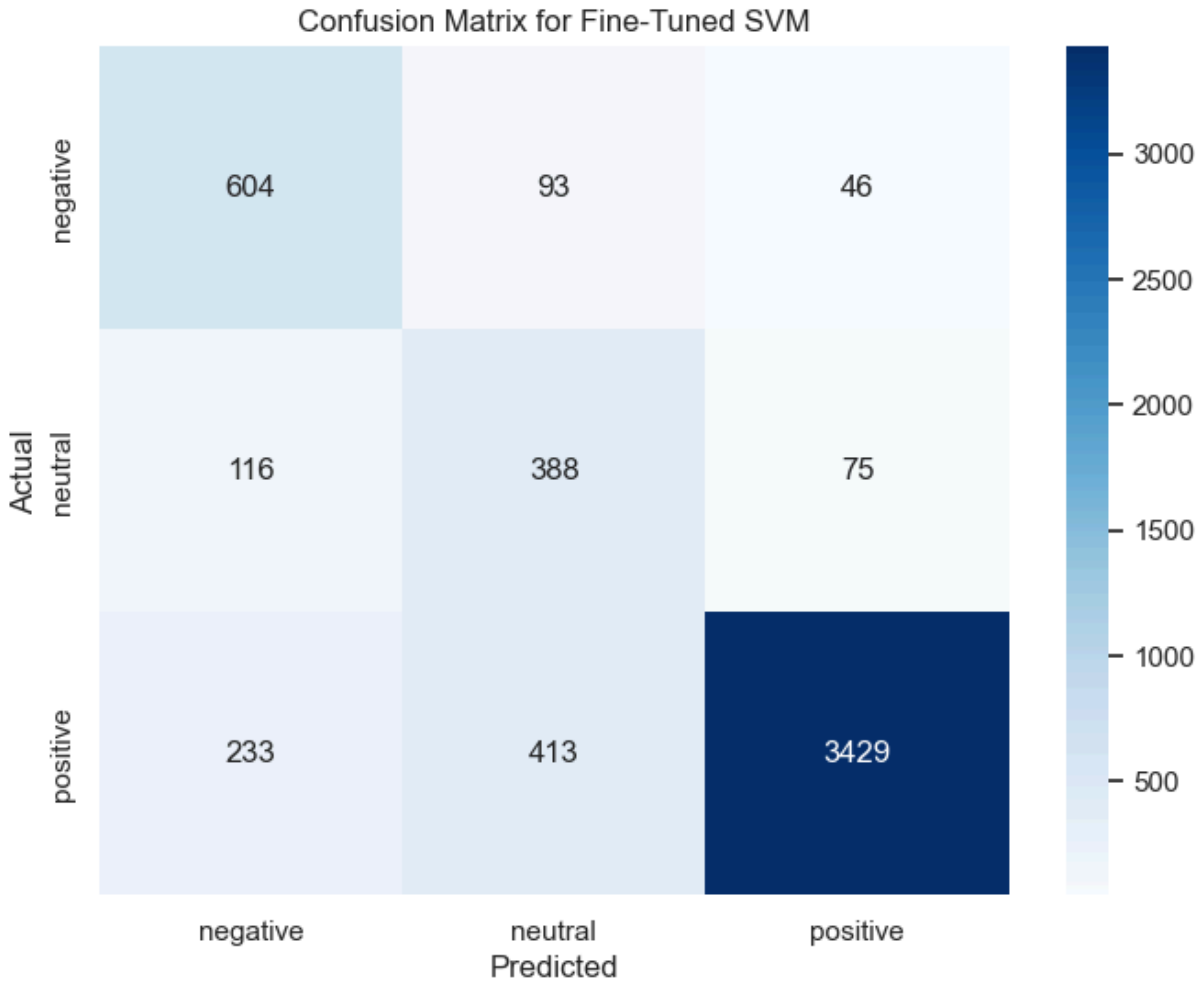
In [86]:
```
# Define param grid
param_grid = {
    #'svm__C': [0.1, 1, 10, 100], # Regularization strength
    'svm__C': [10],
    #'svm__kernel': ['linear', 'rbf'],         # Types of kernels
    'svm__kernel': ['rbf'],
    #'svm__gamma': ['scale', 'auto', 0.01, 0.1, 1],  # Kernel coefficient for non-l
    'svm__gamma': [0.01],
    #'svm__class_weight': ['balanced', None],  # Handling class imbalance
    'svm__class_weight': ['balanced']
}

# Define the GridSearchCV
grid_search = GridSearchCV(
    svm_pipeline,
    param_grid=param_grid,
    scoring='recall_macro',  # Focus on improving macro-averaged recall
    cv=5,                    # 5-fold cross-validation
    n_jobs=-1                # Use all CPU cores for parallel processing
)

# Fit the grid search to the data
grid_search.fit(X_train, y_train)

# Best parameters and score
print("Best Parameters:", grid_search.best_params_)
print("Best Macro Recall:", grid_search.best_score_)
```

```
Best Parameters: {'svm__C': 10, 'svm__class_weight': 'balanced', 'svm__gamma': 0.01,
'svm__kernel': 'rbf'}
Best Macro Recall: 0.7638240224820454
```

In [87]:
```
# Use the best model from GridSearchCV
best_svm_pipeline = grid_search.best_estimator_

# Predict on the validation set
y_val_pred_svm = best_svm_pipeline.predict(X_val)
```

```
# Generate the classification report
print("Fine-Tuned SVM Classification Report:\n")
print(classification_report(y_val, y_val_pred_svm))

# Generate the confusion matrix
svm_cm = confusion_matrix(y_val, y_val_pred_svm)

# Visualize the confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(svm_cm, annot=True, fmt='d', cmap='Blues', xticklabels=best_svm_pipelin
plt.title("Confusion Matrix for Fine-Tuned SVM")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.savefig("Confusion Matrix Tuned SVM")
plt.show()
```

```
Fine-Tuned SVM Classification Report:

              precision    recall  f1-score   support

    negative       0.63      0.81      0.71       743
     neutral       0.43      0.67      0.53       579
    positive       0.97      0.84      0.90      4075

    accuracy                           0.82      5397
   macro avg       0.68      0.77      0.71      5397
weighted avg       0.86      0.82      0.83      5397
```



Negative Class

- Precision (0.63): When predicting negative sentiment, `63% of the predictions are correct`. This is moderate and suggests some misclassifications into other classes.
- Recall (0.81): The model successfully identifies `81% of actual negative samples`, reflecting significant improvement in recall compared to the baseline models.
- F1-Score (0.71): The balance between precision and recall shows that the model performs fairly well for the negative class.

Neutral Class

- Precision (0.43): The model predicts neutral sentiment less reliably, with only `43% of predictions being accurate.`
- Recall (0.67): The model identifies `65% of the true neutral samples`, showing improved recall but still room for growth.
- F1-Score (0.53): Moderate performance due to lower precision but decent recall.

Positive Class

- Precision (0.97): Extremely high precision indicates that most predictions of positive sentiment are correct.
- Recall (0.84): The model identifies `84% of actual positive samples`, which is strong but slightly lower than precision.
- F1-Score (0.90): Outstanding overall performance for the dominant positive class.

Overall Performance

- Accuracy (0.82): The model classifies `82% of instances correctly`, which is solid overall.
- The fine-tuned SVM model demonstrates strong overall performance, particularly for the positive and negative classes. It has improved recall for the negative and neutral classes, making it more balanced than the baseline.

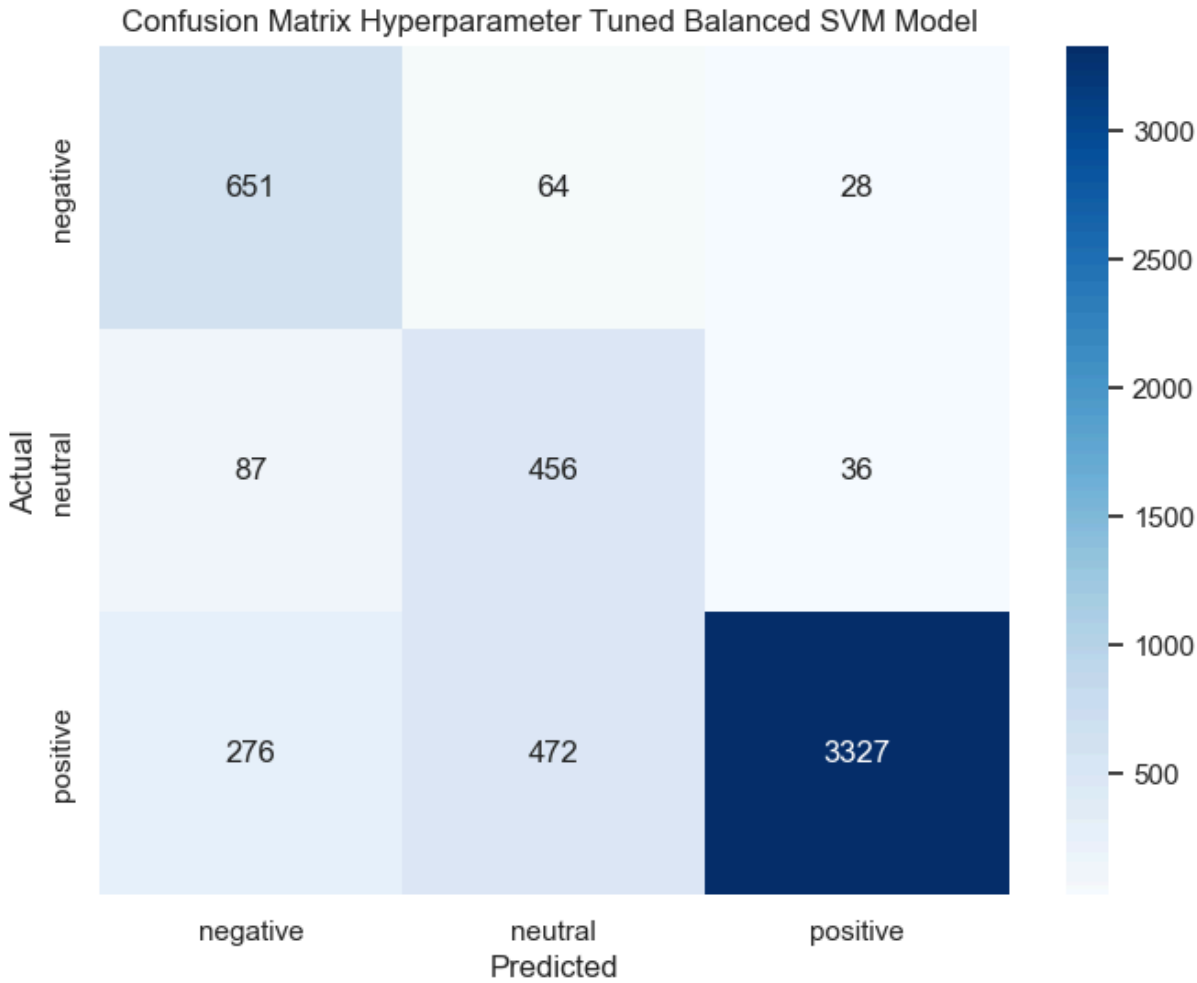
However, further optimization is needed to improve the neutral class performance, particularly precision and recall. Techniques like under-sampling the positive class for more balance classes, could help. In the next model iteration, we will undersample the positive class and use the best params identified as they have shown marked improvement.

3.4 Class Balanced Tuned SVM Model

In [88]:
```
# Separate the classes
negative_class = data[data['sentiment'] == 'negative']
neutral_class = data[data['sentiment'] == 'neutral']
positive_class = data[data['sentiment'] == 'positive']

# Check class sizes
print("Class sizes before undersampling:")
print(f"Negative: {len(negative_class)}, Neutral: {len(neutral_class)}, Positive: {

# Undersample the positive class to match the size of the neutral class
positive_class_undersampled = resample(positive_class,
                                       replace=False,  # Sampling without replaceme
                                       n_samples=4000,  # Undersample the Positive
                                       random_state=42)  # For reproducibility

# Combine the balanced classes
balanced_data = pd.concat([negative_class, neutral_class, positive_class_undersampl

# Shuffle the dataset
balanced_data = balanced_data.sample(frac=1, random_state=42).reset_index(drop=True

# Check class sizes after undersampling
print("Class sizes after undersampling:")
print(balanced_data['sentiment'].value_counts())

# Split into features and target
X_balanced = balanced_data['clean_review']
y_balanced = balanced_data['sentiment']
```

```
# Re-train the fine-tuned SVM model
best_svm_pipeline.fit(X_balanced, y_balanced)

# Evaluate the re-trained model
y_val_pred_balanced = best_svm_pipeline.predict(X_val)

# Classification report and confusion matrix
# Classification report
print("Classification Report (After Undersampling):\n")
print(classification_report(y_val, y_val_pred_balanced))

# Confusion matrix
balanced_cm = confusion_matrix(y_val, y_val_pred_balanced)
plt.figure(figsize=(8, 6))
sns.heatmap(balanced_cm, annot=True, fmt='d', cmap='Blues', xticklabels=best_svm_pi
plt.title("Confusion Matrix Hyperparameter Tuned Balanced SVM Model ")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.savefig("Confusion_Matrix_Balanced_SVM.png")
plt.show()
```

```
Class sizes before undersampling:
Negative: 3715, Neutral: 2894, Positive: 20375
Class sizes after undersampling:
sentiment
positive    4000
negative    3715
neutral     2894
Name: count, dtype: int64
Classification Report (After Undersampling):

              precision    recall  f1-score   support

    negative       0.64      0.88      0.74       743
     neutral       0.46      0.79      0.58       579
    positive       0.98      0.82      0.89      4075

    accuracy                           0.82      5397
   macro avg       0.69      0.83      0.74      5397
weighted avg       0.88      0.82      0.84      5397
```



Positive Class

- Precision: Slight improvement (`0.97 to 0.98`) after undersampling, meaning there were fewer false positives.
- Recall: Decreased slightly (`0.84 to 0.82`), indicating that the model captured slightly fewer true positives from the positive class after undersampling.
- The undersampling caused a small trade-off in recall but improved precision.
- F1-score: A minor drop (`0.90 to 0.89`), reflecting a balance between precision and recall.

Neutral Class

- Precision: Improved marginally (`0.43 to 0.46`).
- Recall: Significant improvement (`0.67 to 0.79`), meaning the model performed better at identifying true neutral instances post-undersampling, a significant boost in recall.
- F1-score: Noticeable improvement (`0.53 to 0.58`).

Negative Class

- Precision: Slight improvement (0.63 to 0.64).
- Recall: Improved significantly (from 0.81 to 0.88), showing better identification of true negatives after undersampling.
- F1-score: A minor improvement (from 0.71 to 0.74).

Overall, while accuracy stayed constant at 82%, the macro average scores improved, highlighting better balance across classes. This is especially critical when addressing class imbalance.

The undersampling approach prioritized fairness across classes, slightly sacrificing performance on the majority class (positive) for better results in the minority classes.

3.5 Best Model Results

The `undersampling` strategy effectively addressed class imbalance, in the `Hyperparameter Tuned SVM Model` leading to improved fairness across all classes:

- Positive Class: Precision improved slightly (0.97 → 0.98), reducing false positives. However, recall dropped marginally (0.84 → 0.82), indicating fewer true positives captured. This trade-off is reflected in a minor decrease in the F1-score (0.90 → 0.89).

- Neutral Class: The model saw significant gains in recall (0.67 → 0.79) and modest improvements in precision (0.43 → 0.46), resulting in a noticeable F1-score boost (0.53 → 0.58). This highlights better recognition of true neutral instances.

- Negative Class: Recall improved significantly (0.81 → 0.88), with a slight increase in precision (0.63 → 0.64), leading to a modest F1-score rise (0.71 → 0.74). This demonstrates enhanced identification of negative cases.

While overall accuracy remained steady at 82%, the macro average scores improved, emphasizing better balance and fairness across classes. The confusion matrix confirms these improvements, particularly in the neutral and negative classes. Undersampling successfully reduced bias toward the majority class (positive), making the model more equitable across the dataset.

We will save and deploy this model for current use as we continue to use more advanced modeling techniques such as LSTMs to further improve performance

3.6 Save the Best Performing Model

In [89]:
```
best_svm_pipeline
```

Out[89]:

```
▸ Pipeline
  ▸ TfidfVectorizer
  ▸ SVC
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook. On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [90]:

```
# save best model
joblib.dump(best_svm_pipeline, "svm_pipeline.pkl")

print("SVM pipeline (model + vectorizer) saved successfully!")
```

```
SVM pipeline (model + vectorizer) saved successfully!
```

4.0 Recommendations

4.1 A Deep Dive into Sentiment Distribution

The overwhelming dominance of `positive` sentiments suggests that the majority of viewers are highly satisfied with Prime Video's content. However, delving `DEEPER` into the `neutral and negative` feedback provides valuable opportunities to identify pain points and enhance the overall user experience. We recommend the following strategies to leverage on this:

1. Segment the Sentiments

Categorize the neutral and negative feedback into specific themes, such as content quality, casting, genre preferences, technical issues (e.g., streaming quality), or user interface complaints.

2. Analyze Key Pain Points

Pinpoint recurring issues within each category. For example, if "storyline" appears frequently in negative trigrams, investigate feedback related to poorly developed plots or confusing narratives.

3. Prioritize Areas for Improvement

Focus on the most critical pain points that impact overall viewer experience. Use metrics like frequency of mentions or intensity of dissatisfaction to determine priority areas.

4. Develop Targeted Solutions

For each identified pain point, outline specific measures:-

- Improve storytelling quality by collaborating with seasoned writers or directors.
- Address technical complaints by upgrading streaming technology or optimizing the app interface.
- Expand diverse content offerings to appeal to underrepresented audience segments.

5. Leverage Positive Sentiment for Marketing

While addressing neutral and negative feedback, don't neglect the overwhelming positivity. Highlight successful elements (popular genres, titles, or features) in marketing campaigns to attract and retain customers.

6. Close the Feedback Loop

Implement a system to collect ongoing feedback on changes made. For example, update the sentiment analysis models to gauge whether the updates have improved viewer satisfaction.

4.2 Address Sentiment by Genre

1. Leverage Popular Genres

- **Prioritize Promotion and Recommendations:** Highlight popular genres such as Animation, Kids, Fantasy, Fitness, Historical, Talkshow, and Western by featuring them prominently on Prime Video's homepage and in targeted recommendations.
- **Develop Exclusive Content:** Invest in original series or movies in these favored categories to sustain engagement and grow their audience base.
- **Curate Fan Favorites:** Create genre-specific playlists to celebrate popular titles, keeping audiences coming back for more.

2. Address Critical Feedback

- **Investigate Viewer Concerns:** Dive deep into sentiment data for less favorable genres like Horror and Arts. Analyze recurring complaints, such as predictable storylines or limited diversity in offerings, and address them.
- **Enhance Content Quality:** Collaborate with seasoned creators to develop fresh, engaging content that revitalizes these genres.
- **Pilot New Concepts:** Test experimental titles or formats in these categories based on viewer suggestions. For example, blend Horror with other genres like Thriller to see if it resonates better.
- **Engage with Audiences:** Use surveys or focus groups to gather insights directly from viewers about what they'd like to see improved in these genres.

3. Refine Neutral Sentiment Categories

- **Understand Nuanced Feedback:** Analyze the reasons behind neutral sentiment for genres like LGBTQ and Music. Look for patterns such as unrelatable content or lack of representation.

- **Tailor Content Strategies:** Develop programming that aligns with audience expectations—for example, promoting LGBTQ content with authentic stories or introducing fresh concepts in Music, like concert documentaries or artist-focused series.
- **Engage Experts:** Collaborate with creators, consultants, or organizations specializing in these categories to ensure authentic and engaging content.
- **Measure Impact:** Track how new initiatives affect viewer sentiment over time, and adjust strategies accordingly.

5.0 Next Steps

5.1 Model Deployment

Flask-based API

To ensure scalability and real-time sentiment analysis, the Flask-based API serves as the backbone of the deployment process. This lightweight framework supports smooth integration of your trained sentiment analysis model, enabling it to handle incoming data efficiently. Key steps include:

- Building endpoints within Flask to accept user input, process it through the model, and return sentiment results.
- Ensuring scalability by containerizing the application with tools like Docker and deploying it on cloud platforms such as AWS or Azure
- Securing the API by incorporating authentication and data encryption measures.

User Interface (UI)

To enhance accessibility and usability, a user-friendly interface WILL be developed to interact with the model via the Flask API. Key considerations for the UI include:

- **Design & Functionality:** Users should be able to input text, view sentiment results, and explore related insights effortlessly.
- **Real-Time Feedback:** Display sentiment analysis outcomes immediately after user input to make the tool responsive and engaging.
- **Data Visualization:** Integrate graphs or charts to provide users with an overview of sentiment trends, genre-wise breakdowns, or other relevant insights derived from the model.
- **Accessibility:** Optimize the interface for use across devices, including desktops, tablets, and mobile phones, ensuring it caters to diverse user needs.

5.2 Advanced Modeling Techniques

- Explore advanced NLP models like LSTMs, BERT or GPT for real-time sentiment updates and granular analysis.

- Aspect-Based Sentiment Analysis – Identify sentiment for specific aspects (e.g., content quality, streaming experience).
- Emotion Detection – Extend sentiment analysis to classify emotions like joy, anger, and frustration.
- Personalized Recommendations – Suggest movies/shows based on sentiment and user preferences.